



Technische
Universität
Braunschweig

BACHELORARBEIT

Algorithmische Methoden zu gradbeschränkten Triangulierungen

Fabian Alich

5310894

Algorithmik

Institut für Betriebssysteme und Rechnerverbund
Abteilung Algorithmik

Erstprüfer

Prof. Dr. Sándor Fekete

Institut für Betriebssysteme und Rechnerverbund
Abteilung Algorithmik

Zweitprüfer

Prof. Dr. rer. nat. Roland Meyer

Institut für Theoretische Informatik

Betreuer

Dr. Phillip Keldenich

Michael Perk

18. August 2025

Statement of Originality

This thesis has been performed independently with the support of my supervisor/s. To the best of the author's knowledge, this thesis contains no material previously published or written by another person except where due reference is made in the text.

Braunschweig, 18. August 2025

Aufgabenstellung

Eine Triangulierung einer Punktmenge ist die Zerlegung ihrer konvexen Hülle in nicht überlappende Dreiecke. Triangulierungen spielen eine zentrale Rolle in der algorithmischen Geometrie und finden vielfältige Anwendungen, etwa in der Computergrafik und geographischen Informationssystemen (GIS). Im Rahmen seiner Bachelorarbeit soll Herr Alich das neuartige kombinatorische Optimierungsproblem Degree-Constrained Triangulations (DCT) untersuchen. Bei diesem Problem geht es darum, für eine gegebene Punktmenge eine Triangulierung zu finden, bei der – zusätzlich zu den klassischen Anforderungen – die Knotengrade (also die Anzahl der angrenzenden Dreiecke bzw. Kanten pro Punkt) gewissen Beschränkungen unterliegen. Ziel der Arbeit ist es, geeignete Verfahren zur Lösung dieses Problems zu entwickeln, zu implementieren und zu evaluieren. Dabei können neben exakten Verfahren (z.B. auf Basis von Integer Programming, Constraint Programming oder Branch-and-Bound) auch heuristische Ansätze (z.B. Greedy- oder Metaheuristiken) betrachtet werden. Von besonderem Interesse sind außerdem Methoden zur Fixierung oder zum Ausschluss bestimmter Kanten, um die Lösungsräume gezielt einzuschränken sowie die Generierung von leichten und schweren Instanzen für das Problem.

Abstract

Your abstract gives a short introduction to your thesis (context), the problem description, and a brief description of your results.

You may want to include a german abstract, too.

Inhaltsverzeichnis

1. Introduction	3
1.1. Das Problem	3
1.2. Results	4
1.3. Related Work	4
1.3.1. Delaunay Triangulations	5
1.3.2. Kantenflips in Triangulations	6
1.3.3. Degree constrained minimum spanning tree	6
1.4. Overview	11
2. Definitionen	13
3. Theoretischer Hintergrund	15
3.1. Solver	15
3.2. Kanten Formulierung	15
3.2.1. Implementierung der Kanten Formulierung	18
3.3. Dreieck Formulierung	22
3.3.1. Implementierung der Dreiecksformulierung	23
3.4. Optimierungsproblem	24
3.4.1. Flips	24
3.4.2. OrTools mit Durchschnitts Abweichung	25
3.4.3. OrTools mit Minimierter Maximaler Abweichung	26
3.5. <i>Conflict-driven clause learning</i>	26
3.5.1. <i>Unit-Klauseln</i>	27
3.5.2. Konflikte	27
3.6. Instanzgenerierung	28
3.6.1. Ja - Instanzen	28
3.6.2. Nein-Instanzen	30
3.7. Theoretische Mehrere Lösungen	31
4. Auswertung	33
4.1. Versuchsumgebung	33
4.2. Permutation	34
4.3. Mehrmalige Durchläufe	34
4.4. SAT Gradbeschränkung	34
4.5. SAT Solver Vergleich	35
4.6. Grund Solver	36
4.7. Kanten vorab berechnen	38

Inhaltsverzeichnis

4.8. Ja/Nein getrennt	38
4.9. Best Solver	38
4.10. Zielfunktion	39
4.11. <i>Propagation</i> CDCL	40
4.11.1. Visualisierung	40
4.11.2. Implementierung	41
4.11.3. Experiment	41
4.12. Instanzen Schwierigkeit	42
4.12.1. Anzahl Lösungen	43
4.12.2. Grad Verteilung	43
4.12.3. Kantenlängen Verteilung	44
4.12.4. Schwierigkeit Zusammenfassung	44
5. Conclusion (and Future Work)	47
A. SAT Degree Auswertung	51
B. Alle Parameter	53
C. Anzahl Lösungen	57
D. Zielfunktion	59

Abbildungsverzeichnis

1.1. Eine Ja-Instanz und eine Nein-Instanz	4
1.2. Delauney Beispiel	5
1.3. Beispiel für <i>edge move</i> , <i>edge rotation</i> und <i>edge slide</i> aus [5, p. 70]	7
2.1. Ein Kantenflip in einer Triangulation	14
3.1. Gesonderte Betrachtung von Knoten auf der Hülle mit Grad zwei	17
3.2. zwei Beispielinstanzen an denen Kanten ausgeschlossen werden können . .	18
3.3. Kantenausschluss durch Schnittkanten	18
3.4. Ein Vollständiger Graph mit allen möglichen Triangulierung.	19
3.5. Ein Baum welcher die Möglichen Entscheidungen eines SAT-Solvers dar- stellt. Jeder Ast entspricht Belegungen der Literale	27
3.6. Visualisierung der fünf verschiedenen Instanztypen.	30
3.7. Eine Ja-Instanz mit zwei möglichen Lösungen. Die Zahlen an den Knoten geben hierbei den Sollgrad an. Die roten kanten sind jene die in den Lösungen unterschiedlich sind.	32
4.1. Vergleich von Kardinalitätscodierungen für SAT-Solver	35
4.2. Eval3 Gesamt	36
4.3. Eval1 Gesamt	37
4.4. eval11 move	39
4.5. Eine Teilinstanz bei einem <i>Propagation</i> Aufruf. Blau auf false gesetzt, grün auf true. Grüner Knoten Sollgrad erfüllt. Zahl in Knoten ist Sollgrad. TODO schön aufschreiben mit finalem Bild und erklären das von rand ausgehend fertig	40
4.6. Anzahl propergation cdcl	42
4.7. zwei verschiedene Lösungen für eine Instanz	43
4.8. Eval8 Anzahl Lösungen Greedy	44
4.9. Eval8 Kanten verteilung	45
A.1. Die <i>exact</i> Methode für die Gradbeschränkung in einem SAT Solver, wurde mit verschiedenen Kardinalitätsbedingung getestet.	51
A.2. Die <i>atleast</i> Methode für die Gradbeschränkung in einem SAT Solver, wurde mit verschiedenen Kardinalitätsbedingung getestet.	52
B.1. Eval1 Dreiecke	53
B.2. Eval1 Sat	54
B.3. Eval1 OrTools	54

Abbildungsverzeichnis

B.4. Eval1 Gurobi	55
D.1. eval11 36	59

Tabellenverzeichnis

C.1. In dieser Tabelle werden die Anzahl der Lösungen für die verschiedenen Ansätze dargestellt. In der ersten Spalte werden die Anzahl der Knoten der Instanz angegeben. Hierbei wurde jede Knotengröße zwei mal getestet. In den anderen Spalten sind jeweils die Werte für eine Instanz gegeben. Die Werte -1 bedeuten, dass nach einer Stunde ein Timeout erreicht wurde und somit keine Lösung gefunden wurde konnte. Auffällig ist, dass es nur eine einzige Instanz gibt, welche zwei Lösungen besitzt.	57
--	----

FA: Passen die ersten Titel so? Bin sehr unsicher wie ich die zusammenfassen soll.

1. Introduction

FA: Sollte ich hier gradbeschränkte Triangulierung irgendwie abkürzen?

Triangulierungen sind ein wichtiger Bestandteil der algorithmischen Geometrie. Eine Triangulierung liefert eine Zerlegung der konvexen Hülle einer Punktmenge in Dreiecke. In dieser Arbeit wird das Problem um eine weitere Bedingung erweitert. Für jeden Knoten wird ein Sollgrad vorgegeben.

Die gradbeschränkte Triangulierung untersucht, ob es für eine gegebene Punktmenge eine Triangulierung gibt, bei der jeder Knoten einen vorgegebenen Grad besitzt. Diese Erweiterung der Triangulierung ist nicht nur theoretisch interessant, sondern hat auch praktische Anwendungen. So kann zum Beispiel in Sensornetzwerken die Verbindungs-dichte zwischen Knoten reguliert werden. Auch in Logistikproblemen können vorgegebene Kapazitäten an Knoten berücksichtigt werden.

Das Ziel dieser Arbeit ist es, Verfahren zur Lösung von gradbeschränkten Triangulierungen zu untersuchen. Neben der Betrachtung als Entscheidungsproblem, bei dem Methoden wie Integer Programming, Constraint Programming oder SAT Solver verwendet werden, wird das Problem auch als Optimierungsproblem betrachtet. Ein Schwerpunkt liegt auf dem Fixieren und Ausschließen von Kanten, um die Suche nach Lösungen zu beschleunigen. Im gleichen Zusammenhang werden verschiedene Methoden zur Instanzgenerierung vorgestellt.

Die Arbeit beginnt mit grundlegenden Definitionen und relevanten theoretischen Grundlagen. Anschließend werden verschiedene Lösungsansätze vorgestellt und eine mögliche Implementierung beschrieben. Mit dieser Implementierung werden dann die verschiedenen Lösungsansätze evaluiert. Der Fokus liegt dabei auf der Laufzeit und dem Finden des besten Lösungsansatzes. Abschließend werden die generierten Instanzen hinsichtlich ihrer Schwierigkeit untersucht.

1.1. Das Problem

Das in dieser Arbeit betrachtete Problem sind die gradbeschränkten Triangulierungen. Das Problem untersucht, ob für eine gegebene Punktmenge eine Triangulierung existiert, bei der jeder Knoten einen vorgegebenen Grad besitzt. In Abbildung 1.1 sind zwei Instanzen mit Lösungen dargestellt. Die Zahlen an den Knoten bezeichnen dabei die Sollgrade. Die obere Instanz ist eine Ja-Instanz, da die Sollgrade mit der Triangulierung eingehalten werden können. Bei der unteren Instanz ist hingegen mit den gegebenen Sollgraden keine Triangulierung möglich.

Praktisch kann das Problem zum Beispiel in der Computergrafik genauer in der Erzeugung von Netzen für die Modellierung von Oberflächen genutzt werden. Durch

1. Introduction

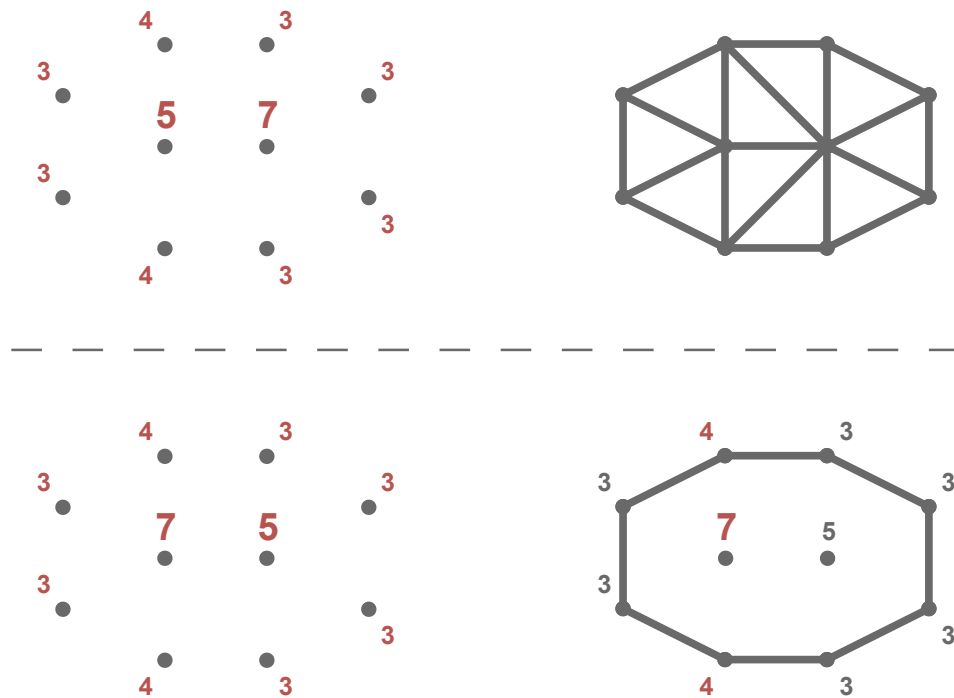


Abbildung 1.1.: Eine Ja-Instanz (oben) und eine Nein-Instanz (unten). Die Zahlen an den Knoten sind die Sollgrade. Bei der Nein-Instanz können die Sollgrade mit einer Triangulierung nicht eingehalten werden.

den Sollgrad kann dann die Dichte oder der Platzverbrauch in bestimmten Bereichen gezielt gesteuert werden. Analog kann in der Nachrichtentechnik eine gradbeschränkte Triangulierung verwendet werden, um die Verbindungsdichte zwischen Knoten in einem Sensornetzwerk zu regulieren. Ebenso kann die Gradbeschränkung Triangulierung genutzt werden wenn Güter von einem Ort an einen anderen Transportiert werden sollen. Der Sollgrad könnte angeben wie viele Güter an jedem Ort vorhanden sind und weiterbewegt werden sollen.

1.2. Results

FA: muss/soll ich hier was angeben? Weil ich ja jetzt nicht Das Ergebnis habe.

1.3. Related Work

In diesem Abschnitt werden verschiedene wissenschaftliche Arbeiten aus diesem Themenbereich betrachtet. Dies dient dazu, einen Überblick über den aktuellen Entwicklungsstand zu erhalten. Dazu werden drei Bereiche begutachtet. Der erste Bereich behandelt die

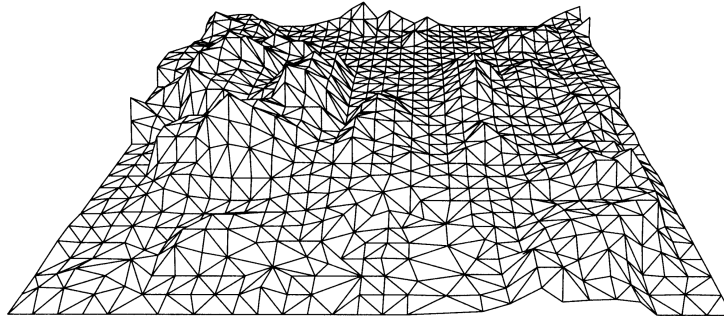


Abbildung 1.2.: Ein Beispiel für Höhendaten durch eine Triangulierung aus [8, p. 183]

Delaunay Triangulierung. Diese zählt zu den bekanntesten Triangulations und bietet einen grundlegenden Einblick in das Thema Triangulierung. Der zweite Bereich umfasst *Flips in Triangulations*. Hier wird untersucht, wie sich Triangulierung durch lokale Änderungen gezielt modifizieren lassen. Der dritte und letzte Bereich befasst sich mit dem Problem des *degree constrained minimum spanning tree*. Dabei werden insbesondere Ansätze zur Einhaltung von Grad Beschränkungen betrachtet.

Es ist noch zu erwähnen, dass Sharir et. al. [19] den minimalen und maximalen Grad in einer Triangulierung untersucht haben. Darüber hinaus haben Datta et. al. [6] und Gewali et. al. [11] sich mit Triangulierung beschäftigt, bei denen jeder Knoten einen geraden Grad hat. Da diese beiden Themenbereiche keinen direkten Mehrwert für die vorliegende Arbeit bieten, werden sie hier lediglich der Vollständigkeitshalber namentlich erwähnt.

1.3.1. Delaunay Triangulations

Als Grundlage für den Algorithmus kann die *Delaunay-Triangulierung* betrachtet werden. Diese liefert eine Triangulierung, die möglichst gleichschenklige und gleichwinklige Dreiecke verwendet [17]. Dies wird erreicht, indem der minimale Winkel maximiert wird. Im Buch von de Berg et al. [8] wird diese Variante der Triangulierung ausführlich behandelt.

Triangulations eignen sich durch ihre Struktur besonders gut zur Darstellung von Höhendaten im zweidimensionalen Raum (siehe Abbildung 1.2). Dabei kann ausgenutzt werden, dass Dreiecke unterschiedliche Innenwinkel und Kantenlängen besitzen.

Das Grundproblem besteht darin, dass die Höhendaten nur an bestimmten Punkten bekannt sind und die restlichen Werte daher geschätzt werden müssen. Dies kann durch die Delaunay-Triangulierung gut gelöst werden, da die Dreiecke aufgrund ihrer gleichschenkligen Eigenschaft visuell geeignet sind, die fehlenden Daten zu ergänzen.

Es werden auch Formeln für die Anzahl der Kanten $|E|$ und Dreiecke $|F|$ in einer Triangulierung angegeben. Für eine Punktemenge mit n Punkten und k Punkten auf der

1. Introduction

konvexen Hülle gilt:

$$|E| = 3n - 3 - k$$

$$|F| = 2n - 2 - k$$

PK: Flips in Triangulationen?

FA: so verständlicher? könnte sie auch Delauney flips nennen. In der Zitierten arbeit werden sie auch Flips genannt

1.3.2. Kantenflips in Triangulations

Kantenflips oder kurz Flips sind Möglichkeiten, um Kanten in einer Triangulierung auszutauschen. Wie oben erwähnt, müssen Triangulierung eine feste Anzahl an Kanten besitzen [8]. Dementsprechend können keine Kanten hinzugefügt oder entfernt werden, um den Grad einer Triangulierung zu verändern. Daraus folgt, dass für jede entfernte Kante eine andere Kante eingefügt werden muss. Eine besondere Art dieser Umstrukturierung von Kanten sind sogenannte Flips. Hurtado et al. [12] zeigten, dass jede Triangulierung mindestens

$$\frac{n - 4}{2}$$

flipbare Kanten besitzt, wobei n die Anzahl der Knoten bezeichnet. Laut der wissenschaftliche Arbeit *Flips in planar graphs* [5] lässt sich jede mögliche Triangulierung in jede beliebige andere Triangulierung überführen, indem die notwendigen Flips durchgeführt werden. Daraus folgt, dass, wenn eine Gradbeschränkte Triangulierung möglich ist, diese mit den nötigen Flips konstruiert werden kann. Dieser Ansatz wird in Abschnitt 3.4.1 nochmal näher betrachtet.

Eine weitere Art von Flips sind sogenannte k -Flips. Diese stellen eine zusätzliche Möglichkeit dar, Kanten innerhalb einer Triangulierung zu verschieben. Dabei werden k Kanten entfernt und durch k neue Kanten ersetzt. Normale Flips entsprechen somit 1-Flips. Der Vorteil von k -Flips liegt darin, dass mit einem Schritt größere strukturelle Veränderungen möglich sind.

Drei beispielhafte Flips sind in Abbildung 1.3 dargestellt. Zum einen der *edge move*, hierbei handelt es sich um das einfache Entfernen und anschließende Einfügen einer Kante. Etwas stärker eingeschränkt ist die *edge rotation*, bei der ein Endpunkt der Kante unverändert bleibt. Die Rotation erfolgt also nur um einen Knoten. Die dritte Variante, der *edge slide*, schränkt die *edge rotation* weiter ein. Hier darf der veränderte Knoten nur ein Nachbar des ursprünglichen Knotens sein.

1.3.3. Degree constrained minimum spanning tree

Eine Gradbeschränkung gibt es auch beim Minimum Spanning Tree. Hier hat jede Kante ein vorgegebenes Gewicht, welches im Spanning Tree minimiert werden soll. Das Besondere am *degree constrained minimum spanning tree* ist, dass ein Minimum Spanning

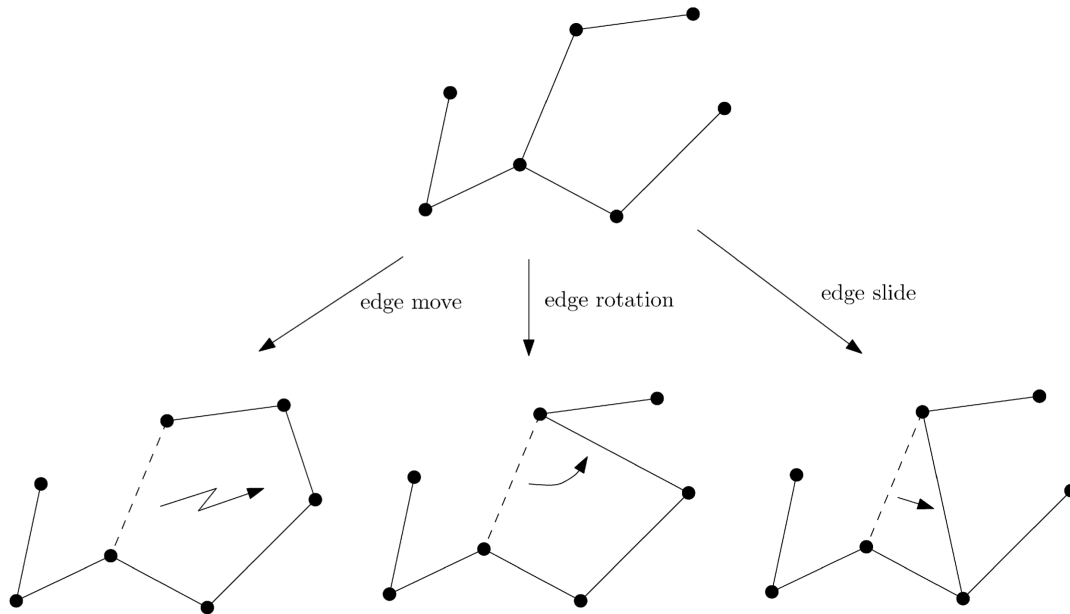


Abbildung 1.3.: Beispiel für *edge move*, *edge rotation* und *edge slide* aus [5, p. 70]

Tree erzeugt werden soll, welcher an jedem Knoten einen maximalen Grad von d hat. Dabei wird d global für das Problem vorgegeben.

Es wurden verschiedene Ansätze von Joshua Knowles und David Corne [13] vorgestellt, welche das Problem lösen sollen. Der erste Ansatz ignoriert zu Beginn den Aspekt der Minimalität des Problems. Es werden Kanten hinzugefügt, welche die Gradbeschränkung nicht verletzen. Im weiteren Verlauf wird versucht, die Anzahl der Kanten zu minimieren.

Ein anderer Ansatz nutzt aus, dass der minimale Spannbaum ohne Gradbeschränkung in polynomialer Zeit gelöst werden kann. Dieser Ansatz erzeugt zuerst einen minimalen Spannbaum, welcher die Gradbeschränkung ignoriert. Wenn eine Lösung gefunden wurde, werden den Kanten neue Gewichte zugeteilt. Diese neuen Gewichte werden mit folgender Formel berechnet,

$$weight + fault \cdot max \cdot \frac{(weight - min)}{(max - min)}$$

wobei *weight* das aktuelle Gewicht der Kante ist. Die Variable *fault* hat den Wert, $\{0, 1, 2\}$ abhängig davon, wie viele Knoten an der Kante gerade die Gradbeschränkung verletzen. Die Variablen *min* und *max* bezeichnen das kleinste beziehungsweise größte Gewicht aller Kanten. Danach wird erneut ein minimaler Spannbaum gebildet, wobei durch das neue Gewicht nun Kanten bevorzugt werden, welche keine Gradbeschränkung verletzen. Diese beiden Schritte können so lange wiederholt werden, bis eine valide Lösung gefunden wurde oder eine maximale Anzahl an Iterationen erreicht wurde. Das Problem bei diesem Ansatz ist, dass Lösungen fälschlicherweise als *nicht möglich* bewertet werden können aufgrund der maximalen Iterationen.

Ein weiterer Ansatz wird als *Hillclimbing* bezeichnet. Dieser löst das Problem des

1. Introduction

degree constrained minimum spanning tree nicht, wird jedoch zur Suche nach neuen Knoten in einem anderen Algorithmus verwendet. In diesem Ansatz werden lokal neue Knoten gesucht, die bessere Ergebnisse liefern. Dies könnte auf die *Degree constrained Triangulierung* angewendet werden, um neue Kanten zu finden. Der Ansatz erzeugt keine eigene Lösung, sondern verbessert eine bestehende. Zunächst wird ein zufälliger Knoten aus einer Lösung ausgewählt. Danach werden weitere mögliche Knoten in der Nähe des ursprünglichen Knotens gesucht. Bei diesen Knoten wird überprüft, ob sie gleich gute oder bessere Ergebnisse liefern. Falls dies der Fall ist, wird der Knoten ausgetauscht, andernfalls wird der gefundene Knoten ignoriert. Dieser Ansatz soll lokal gute Ergebnisse liefern, bringt jedoch aufgrund seines *greedy*-Verhaltens keine globalen Verbesserungen. Eine mögliche Verbesserung der lokalen Problematik stellt *multistart hillclimbing* dar. Bei diesem Verfahren wird nur ein Punkt in der Nähe getestet. Ist dieser nicht gleich oder besser, wird ein neuer zufälliger Punkt ausgewählt.

Der nächste Ansatz wird *simulated annealing* genannt. Auch hier werden nur neue Knoten gesucht, um bestehende Lösungen zu verbessern. Das Ziel des Ansatzes ist es, das Problem mit einer globalen Verbesserung zu lösen, indem zu Beginn viele Veränderungen zugelassen werden und im Verlauf immer genauere Verbesserungen erfolgen. Auch hier wird ein zufälliger Knoten ausgewählt und es werden weitere mögliche Knoten in der Nähe gesucht. Bei diesem Ansatz werden jedoch neue Knoten gemäß der folgenden Formel übernommen:

$$p\{\text{accept } j\} = \begin{cases} 1 & \text{if } f(j) \leq f(i) \\ \exp\left(\frac{f(i)-f(j)}{c}\right) & \text{sonst} \end{cases}$$

Dabei ist $p\{\text{accept } j\}$ die Wahrscheinlichkeit, dass ein Knoten übernommen wird. Die aktuelle Lösung ist i und die mögliche neue Lösung ist j . Die Funktion f gibt hierbei die Kosten der Lösung an. Der Parameter c wird zu Beginn sehr hoch gesetzt und mit der Zeit verringert. Die Veränderung von c bewirkt die gewünschte Änderungsrate. Der Ansatz endet, wenn ein finales c_f erreicht wird, also wenn $c = c_f$.

Der letzte Ansatz wird erst am Ende vorgestellt und beschreibt das übliche Verfahren eines Evolutionsalgorithmus. Dabei werden verschiedene Lösungen erzeugt und bewertet. Die besten Lösungen werden ausgewählt und auf Basis dieser Lösungen werden durch Mutation neue Lösungen generiert.

Krishnamoorthy et.al. [15] hat Evolutionsalgorithmen noch mal genauer betrachtet. Der erste Schritt ist es eine Codierung zu wählen, welche nur *richtige* Spannbäume darstellen kann. Sonst kann es passieren, dass von den erstellten Lösungen viele nicht genutzt werden können und somit nicht relevant sind. Wenn eine passende Codierung gewählt wurde, können dann nur noch der Grad und das Gewicht betrachtet werden.

Eine Möglichkeit wie Mutationen stattfinden können, ist die *Crossover* Methode. Dabei existieren zwei *Eltern* Lösungen P_1 und P_2 . Aus diesen werden

$$2 \cdot (n - 2)$$

verschiedene *Kinder* erzeugt, da es $n - 1$ Veränderungsmöglichkeiten gibt. Anschließend werden die *Kinder* bewertet und sortiert. Das beste *Kind*, das P_1 am ähnlichsten ist,

wird ausgewählt. Ist dieses *Kind* besser als P_1 , ersetzt es P_1 . Für P_2 wird ein *Kind* nach dem gleichen Verfahren ausgewählt.

Eine weitere Möglichkeit ist die *Breeding* Methode. Dabei existiert ein Pool aus Lösungen. Zufällig werden Paare gebildet, die jeweils zwei *Kinder* erzeugen. Von diesen *Kindern* ersetzt das bessere *Kind* das schlechtere *Elternteil*. Die Größe des Pools kann so angepasst werden, dass alle möglichen Veränderungen berücksichtigt werden.

Beide Methoden sollen im Durchschnitt alle 10 Generationen die beste Lösung mit allen anderen Lösungen verglichen haben. Als mögliche Iterationsanzahl wird folgende Formel verwendet,

$$\frac{50 \cdot n}{\bar{b}}$$

dabei bezeichnet $\bar{b}$ die durchschnittliche Gradbeschränkung. Diese Formel sorgt dafür, dass große Instanzen mehr Iterationen erhalten, während leichtere Instanzen (mit höherem Grad) schneller bearbeitet werden.

Sandor P. Fekete et al [10] betrachten ebenfalls das *degree constrained minimum spanning tree* problem. Hierbei wird ein *Flow based Algorithmus* verwendet. Da dieser Ansatz wahrscheinlich nicht gut auf Triangulations übertragbar ist, wird er hier nur kurz behandelt.

Im praktischen Teil des Papers werden zwei Algorithmen vorgestellt. Diese erhalten einen minimalen Spannbaum und verändern ihn so, dass er eine Gradbeschränkung einhält. Dabei wird darauf geachtet, dass sich das Gewicht des Spannbaums nicht stark verschlechtert. Für diese Algorithmen werden Hauptoperationen namens *Adoptions* eingeführt. Diese funktionieren ähnlich wie Flips nur für einen Spannbaum. Es handelt sich also um Operationen, die eine legale Veränderung bewirken. Die beiden Algorithmen liefern eine Liste von Adoptionen, welche die gewünschte Änderung erzeugen. Der erste Algorithmus nutzt eine fraktionale Lösung und eine Greedy Strategie, um die Adoptionen zu finden. Der andere Algorithmus beschränkt die betrachteten Kanten auf jene des gegebenen Spannbaums. So können die Algorithmen in polynomialer Zeit eine Lösung finden, die eine Gradbeschränkung einhält.

Eine Abwandlung des minimalen Spannbaums mit Gradbeschränkung wurde von Ana Maria de Almeida et al. [7] untersucht. Dabei erweitern sie das Problem um eine Funktion, welche für jeden Knoten einen Mindestgrad vorgibt. Dieses Problem wird als *Min-Degree Constrained Minimum Spanning Tree* bezeichnet.

FA: ist das NP-schwer so hier richtig, fühlt sich komisch an

Es wurde das *k-Dimensional Matching Problem* verwendet, um zu zeigen, dass das *Min-Degree Constrained Minimum Spanning Tree* Problem NP-schwer ist. Zu erwähnen ist, dass dies in der dritten Dimension nicht gezeigt wurde. Allerdings soll das Problem schwieriger sein als das *Degree Constrained Minimum Spanning Tree*, obwohl auch dieses Problem NP-schwer ist.

Auch in diesem Fall wird das Problem mit einer *Flow based Formulierung* betrachtet. Dabei werden zwei Formulierungen angegeben. Eine gerichtete und eine ungerichtete Formulierung. Die gerichtete Formulierung soll hierbei effizienter Ergebnisse liefern. Die Formulierungen wurden mit einem *Branch and Bound* Algorithmus getestet. Dabei wurde

1. Introduction

festgestellt, dass es bei der gerichteten Formulierung relevant ist, welcher Knoten als Wurzelknoten ausgewählt wird. Es konnte jedoch kein einheitlich optimaler Wurzelknoten für die Formulierungen festgestellt werden. Des Weiteren wurde häufig festgestellt, dass die Relaxierung keine guten Schranken lieferte. Dies konnte damit erklärt werden, dass die zugehörige Formel für den minimalen Spannbaum ohne Gradbeschränkung gleiche Ergebnisse lieferte.

1.4. Overview

If needed, write down where to find everything.

2. Definitionen

MP: Schreib am Kapitelanfang immer ein zwei Einführungssätze, was hier passiert. Du kannst auch zwischen den Definitionen ein paar Überleitungen schreiben, damit sich das nicht liest wie ein Skript.

FA: Passen die Übergänge? Bin unsicher was du hier erwartest.

In diesem Kapitel werden die Definitionen vorgestellt, die für diese Arbeit benötigt werden. Dabei werden insbesondere grundlegende Begriffe erläutert, die in den folgenden Kapiteln vorausgesetzt werden. Zu Beginn werden Überschneidungen zwischen Kanten betrachtet.

Definition 2.1 (Überschneidung). Zwei Kanten gelten als überschneidend, wenn sie sich in der Ebene kreuzen. Im Folgenden bezeichnet die Funktion $I : (\mathbb{Q}^2 \times \mathbb{Q}^2) \rightarrow \mathcal{P}(\mathbb{Q}^2 \times \mathbb{Q}^2)$ die Menge aller Kanten, die eine gegebene Kante schneiden.

Im nächsten Schritt wird die Überschneidung von Dreiecken definiert, da diese in späteren Ansätzen relevant werden.

Definition 2.2 (Überschneidung Dreiecke). Zwei Dreiecke gelten als überschneidend, wenn sie mindestens einen inneren Punkt gemeinsam haben. Das bedeutet, dass sich zwei Außenkanten kreuzen und sich nicht nur berühren dürfen. Im Folgenden bezeichnet die Funktion $I : (\mathbb{Q}^2 \times \mathbb{Q}^2 \times \mathbb{Q}^2) \rightarrow \mathcal{P}(\mathbb{Q}^2 \times \mathbb{Q}^2 \times \mathbb{Q}^2)$ die Menge aller Dreiecke, die ein gegebenes Dreieck schneiden.

Auf Grundlage der Überschneidungen können nun die grundlegenden Begriffe der Triangulierung und der gradbeschränkten Triangulierung definiert werden.

Definition 2.3 (Triangulierung). Eine Triangulierung einer Punktmenge $P \subset \mathbb{Q}^2$ ist ein maximaler kreuzungsfreier Graph auf P . Das bedeutet, es können keine weiteren Kanten hinzugefügt werden, ohne dass sich Kanten schneiden.

Für die gradbeschränkte Triangulierung wird allerdings zuerst der Grad eines Knotens definiert.

Definition 2.4 (Grad). Der Grad eines Knotens ist die Anzahl der anliegenden Kanten. Im Folgenden wird er durch die Funktion $\deg : \mathbb{Q}^2 \rightarrow \mathbb{N}$ angegeben.

Basierend auf dem Grad eines Knotens wird nun die gradbeschränkte Triangulierung eingeführt.

Definition 2.5 (Gradbeschränkte Triangulierung). Die gradbeschränkte Triangulierung hat die gleichen Voraussetzungen wie eine Triangulierung. Zusätzlich gibt es eine Funktion $\deg^* : \mathbb{Q}^2 \rightarrow \mathbb{Q}$. Diese gibt den Grad an, den ein Knoten haben muss.

2. Definitionen

Im Folgenden wird die Operation des Kantenflips beschrieben, die eine wichtige Rolle bei der Umstrukturierung von Triangulierungen spielt.

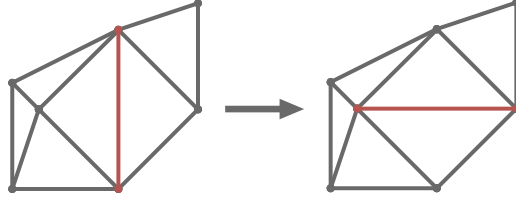


Abbildung 2.1.: Ein Kantenflip in einer Triangulation

Definition 2.6 (Kantenflip). Ein Kantenflip oder kurz Flip ist eine Operation, bei der eine Kante entfernt und eine andere hinzugefügt wird, sodass die Eigenschaften einer Triangulierung erhalten bleiben. In dieser Arbeit wird nur der diagonale Flip betrachtet, bei dem die Diagonale eines konvexen Vierecks durch die andere Diagonale ersetzt wird. Ein beispielhafter Flip ist in Abbildung 2.1 zu sehen.

Im nächsten Schritt wird die konvexe Hülle mit Randpunkten definiert. Da die konvexe Hülle in dieser Arbeit um alle Randpunkte erweitert werden muss.

Definition 2.7 (Konvexe Hülle mit Randpunkten). Die konvexe Hülle einer Punktmenge ist die kleinste konvexe Menge, die alle Punkte enthält. Die Randpunkte sind die Punkte, die auf dem Rand der konvexen Hülle liegen, einschließlich solcher, die nicht an einer Ecke liegen. Die konvexe Hülle inklusive Randpunkte wird durch die Funktion $CH^* : \mathcal{P}(\mathbb{Q}^2) \rightarrow \mathcal{P}(\mathbb{Q}^2)$ definiert. In dieser Arbeit werden bei der konvexen Hülle immer die Randpunkte mit einbezogen.

Abschließend wird die konjunktive Normalform eingeführt, die im weiteren Verlauf für die Benutzung der SAT-Solver benötigt wird.

Definition 2.8 (Konjunktive Normalform). Eine Formel in konjunktiver Normalform (KNF) ist eine logische Formel, die als Konjunktion (UND-Verknüpfung) von Klauseln dargestellt wird, wobei jede Klausel eine Disjunktion (ODER-Verknüpfung) von Literalen ist. Ein Literal ist entweder eine Variable oder ihre Negation. Ein Beispiel für eine KNF ist

$$(x_1 \vee \overline{x_2}) \wedge (x_2 \vee x_3)$$

wobei x_1 , x_2 und x_3 Variablen sind. Eine Negation wird durch das Symbol $\overline{x}$ dargestellt.

3. Theoretischer Hintergrund

In diesem Kapitel werden die verschiedenen Ansätze erklärt mit denen das Problem gelöst werden kann. Im ersten Abschnitt werden die Externen Solver kurz erklärt die in dieser Arbeit genutzt werden. In den beiden darauf folgenden Abschnitten werden zwei Mögliche Ansätze beschrieben die das Problem als Entscheidungsproblem betrachten. Im vierten Abschnitt wird der Problem als Optimierungsproblem betrachtet. Danach wird erklärt wie die Instanzen Generiert werden. Im letzten Abschnitt wird kurz sich mit der Eindeutigkeit der Lösungen beschäftigt.

3.1. Solver

In dieser Arbeit werden drei verschiedene Solver verwendet PySat, OrTools und Gurobi.

PySat ist eine Python-Bibliothek, die eine Sammlung von SAT-Solvern bereitstellt. Diese SAT-Solver entscheiden, ob eine KNF-Formel erfüllbar ist. Dementsprechend kann PySat nur Entscheidungsprobleme, aber keine Optimierungsprobleme lösen. Alle Solver sind in C oder C++ implementiert und es wird eine einheitliche Schnittstelle für alle Solver in Python bereitgestellt. Zusätzlich werden verschiedene Codierungen für Kardinalitäten angeboten. Das bedeutet, man gibt eine Bedingung ein und erhält eine KNF-Formel, die diese Bedingung abbildet. PySat stellt hierfür verschiedene Codierungen mit Implementierung bereit. PySat ist open source und kostenlos.

Der zweite Solver ist OrTools (Google Optimization Tools) von Google. Dieser unterstützt verschiedene Modellierungsarten wie lineare Programmierung (LP), gemischt-ganzzahlige Programmierung (MIP), Constraint Programming (CP) und Routing-Algorithmen. Auch hier sind die Solver in C++ implementiert und es wird eine Python-Schnittstelle bereitgestellt. Mit OrTools können sowohl Entscheidungsprobleme als auch Optimierungsprobleme gelöst werden. Kardinalitäten werden direkt unterstützt und müssen nicht codiert werden. OrTools ist open source und kostenlos unter der Apache License 2.0.

Der letzte Solver ist Gurobi. Es werden ebenfalls Modellierungen wie LP, MIP und quadratische Programmierung (QP) unterstützt. Auch hier wird eine Schnittstelle für Python bereitgestellt. Gurobi ist kostenpflichtig, bietet jedoch eine kostenlose Lizenz für Forschung an.

FA: Ist diese Übersicht in Ordnung, es fühlt sich irgendwie zu wenig technisch an.

3.2. Kanten Formulierung

Es gibt zwei notwendige Bedingung für eine gradbeschränkte Triangulierung.

3. Theoretischer Hintergrund

notwendig Bedingung

Da eine Triangulierung keine Überschneidungen enthalten darf, werden alle Kantenpaare berechnet. Von jedem dieser Paare darf dann nur eine Kante aktiv sein.

Die zweite Bedingung ist die Gradbeschränkung. Für jeden Knoten i wird die Summe der ausgehenden Kanten auf den Sollgrad $\deg^*(i)$ beschränkt. Um dies zu erreichen, wird für jeden Knoten festgelegt, dass die Summe der ausgehenden Kanten dem Sollgrad entspricht.

Eine weitere notwendige Bedingung ist, dass die Kantenanzahl maximal ist. Dies muss nicht explizit als Bedingung angegeben werden da es von der Gradbeschränkung impliziert wird. Es muss nur geprüft werden, ob die Gradbeschränkung in einer Triangulierung erfüllbar ist. Dafür kann die Formel

$$\text{Anzahl Kanten} = 3n - 3 - k$$

aus Abschnitt 1.3.1 verwendet werden. Diese Anzahl der Kanten kann in folgender Formel genutzt werden, um die Gradbeschränkung zu überprüfen.

$$\text{Anzahl Kanten} \cdot 2 \stackrel{!}{=} \sum \deg^*(i)$$

Kanten Fixieren/Ausschluss

Die erste und naheliegendste Bedingung ist die *fixierte Hülle*. Hierbei werden die Kanten der konvexen Hülle immer hinzugefügt. Diese dürfen hinzugefügt werden, da sie keine Schnittkanten besitzen. Andernfalls wäre der Graph nicht maximal und somit keine Triangulierung. Da die Kanten der Hüllen in jeder möglichen Triangulierung vorhanden sind, verletzen sie auch nicht die Gradbeschränkung.

Eine weitere optionale Bedingung ist zuvor berechnete Kanten zu fixieren oder auszuschließen. Eine Möglichkeit um Kanten zu fixieren ist es immer drei aufeinander folgende Knoten der Hülle zu betrachten. Ein mögliches Beispiel ist in Abbildung 3.1 zu sehen. Sollte $\deg^*(v_2) = 2$ sein, muss die Kante e_1 in der Triangulierung genutzt werden. Der Grund dafür ist das alle Schnittkanten von e_1 nicht aktiv sein dürfen, da sie den Grad von v_2 erhöhen würden. Analog dazu kann die Kante e_1 ausgeschlossen werden sollte $\deg^*(v_2) > 2$. Sonst kann der Sollgrad nicht erreicht werden.

Zwei weitere Bedingungen, um Kanten auszuschließen, basieren auf der Teilung der Punktmenge. Es können also alle Kanten betrachtet werden, welche beide Knoten auf der Hülle haben. Die folgenden beiden Bedingungen können genutzt werden, um diese Kanten auszuschließen, sollte eine der Bedingungen wahr werden, so kann die Kante ausgeschlossen werden. Die Bedingungen werden für beide Hälften getestet, welche durch die Teilung entstehen.

$$\begin{aligned} \text{Anzahl Kanten} \cdot 2 &> \sum_{v \in V} \deg^*(v) \\ \forall v \in (V \setminus V_H) : \deg^*(v) &> |V| - 1 \end{aligned}$$

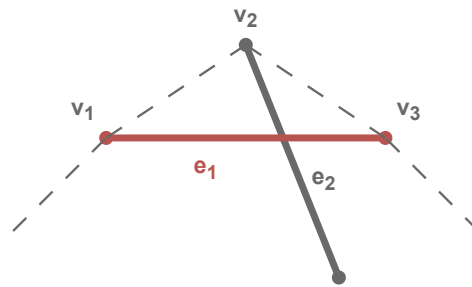


Abbildung 3.1.: Abhängig von dem Sollgrad von v_2 kann entschieden werden, ob e_1 aktiv oder nicht aktiv sein muss. Sollte $\deg^*(v_2) = 2$ sein, dann muss e_1 aktiv sein damit alle Kanten wie e_2 nicht aktiv sein können. Sollte $\deg^*(v_2) > 2$ sein, dann muss e_1 nicht aktiv sein damit der Sollgrad erreicht werden kann.

FA: Anzahl kanten anpassen

Dabei sei V die Menge aller Knoten in der betrachteten Hälfte. Analog sei $V_H \subseteq V$ die Menge der Knoten auf der Hülle.

Zwei Beispiele sind in Abbildung 3.2 zu sehen. Dabei ist Links eine Teilinstanz in welcher die Anzahl der benötigten Kanten (28) größer als die Summe der Grade (27) ist. Auf der rechten Seite ist eine Teilinstanz in welcher der mittlere rote Knoten mit Grad 5 größer ist als die Anzahl der Knoten minus eins (3).

Eine weitere Möglichkeit, Kanten auszuschließen, besteht darin, ihre Schnittkanten zu betrachten. Wenn eine Kante e_1 zu viele Kanten eines Knotens v_1 blockiert, sodass dieser seinen Sollgrad nicht mehr erreichen kann, kann e_1 ausgeschlossen werden.

In Abbildung 3.3 ist zu erkennen, dass e_2 , e_3 und e_4 durch e_1 blockiert werden. Da v_1 jedoch einen Sollgrad von 3 besitzt und nur 5 mögliche Kanten existieren, kann v_1 mit aktiver Kante e_1 seinen Sollgrad nicht erreichen.

Die nächste Bedingung ist nur ein theoretischer Test. Mit den bisher genannten Bedingungen können nicht viele Kanten ausgeschlossen werden. Es kann also nicht betrachtet werden, welchen Einfluss große Mengen von ausgeschlossenen oder fixierten Kanten auf die Laufzeit haben. Daher werden für einzelne Instanzen vor dem Lösen Kanten berechnet, welche sicher ausgeschlossen oder fixiert werden können.

alternative Kanten

Eine andere mögliche Bedingung ist die *alternative Kantenbeschränkung*. Diese wurde von Sándor P. Fekete et.al. [9] übernommen. Diese Bedingung nutzt aus, dass der Graph maximal sein muss. Damit dies der Fall sein kann, muss für jede Kante gelten, dass entweder die Kante aktiv ist oder eine der Schnittkanten aktiv ist. Wie man in Abbildung 3.4 sieht, ist e_1 die ausgewählte Kante und e_2 und e_3 die Schnittkanten.

3. Theoretischer Hintergrund

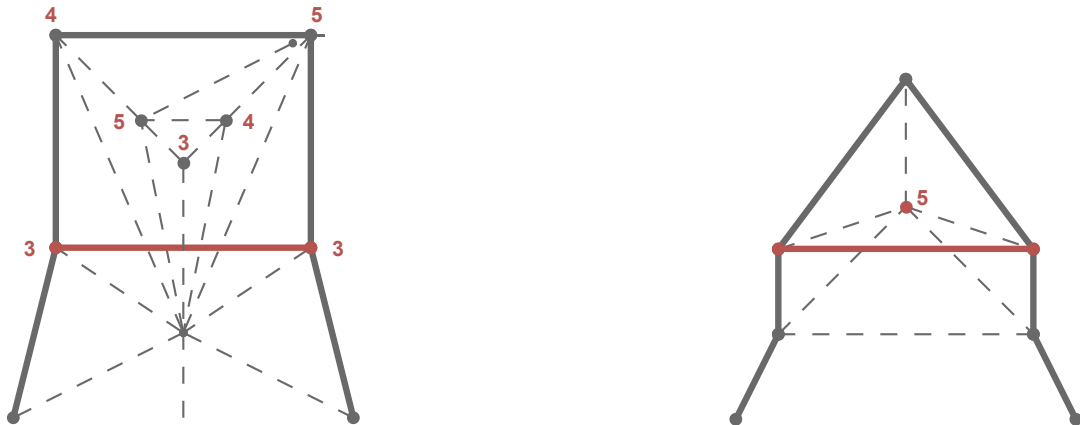


Abbildung 3.2.: Zwei Teilinstanzen, in welcher die rote Kante ausgeschlossen werden kann. Links da die Gradsumme die Anzahl der Kanten unterschreitet und rechts da der mittlere rote Knoten nicht genügend Kanten erzeugen könnte.

Weiter unten im Bild ist zu sehen das alle möglichen Triangulierung mindestens eine der drei Kanten verwendet. Zu beachten ist allerdings das die Anzahl der Schnittkanten nicht gleich eins gesetzt werden darf sondern nur auf mindestens eine. Wie man in Abbildung 3.4 sieht können die Kanten e_2 und e_3 gleichzeitig aktiv sein.

3.2.1. Implementierung der Kanten Formulierung

Für alle Solver wird angenommen das V die Menge aller Knoten und E die Menge aller möglicher Kanten also Kanten die keine Punkte schneiden sei. Für jede Kante $e \in E$ wird eine Bool Variable x_e erstellt. Diese Variable ist wahr wenn die Kante e in der

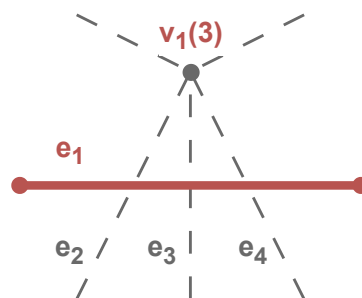


Abbildung 3.3.: In der Abbildung hat der Knoten v_1 einen Sollgrad von 3. Da die Kante e_1 drei von fünf möglichen Kanten blockiert, kann v_1 seinen Sollgrad nicht erreichen. Daher kann e_1 ausgeschlossen werden

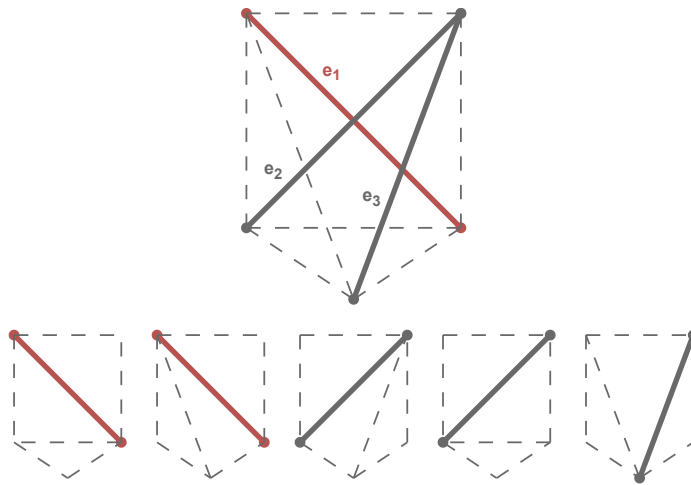


Abbildung 3.4.: Ein Vollständiger Graph mit allen möglichen Triangulierung. Die Kanten e_2 und e_3 sind Schnittkanten von e_1 . Es ist zu sehen das e_1 oder eine der Schnittkanten aktiv sein muss damit eine Triangulierung möglich ist.

Triangulierung enthalten ist.

Graph operationen

Alle Kanten werden mit der Bibliothek *Networkx* verwaltet. Mit dieser können auch die ausgehenden Kanten eines Knotens abgefragt werden. Beim Erstellen der Kanten muss getestet werden, ob die Kanten keinen Knoten schneiden. Dies wird mit der Bibliothek *Shapely* getestet. Bei der Berechnung der Hülle muss beachtet werden, dass alle Knoten, welche auf der Hülle liegen betrachtet werden. Die *convex_hull* Funktion von Shapely liefert nur die Eckpunkte der Hülle. Damit auch Punkte auf einer Geraden zwischen zwei Eckpunkten betrachtet werden, muss nach der Berechnung der Convexen Hülle noch der Schnitt zwischen Convexer Hülle und allen Punkten berechnet werden. In diesem Schnitt sind dann alle Punkte enthalten, welche auf der Hülle liegen.

Schnittkanten

Für die Bestimmung der Schnittkanten wird ein Algorithmus verwendet, der auf dem Konzept des *Rechtsknicks* beziehungsweise *Linksknicks* basiert. Dafür ist es notwendig, dass ein vollständiger Graph betrachtet wird, da angenommen wird, dass zwischen allen Knoten eine Kante existiert. Es wird jede Kante $e_i \in E$, bestehend aus den Knoten $v_{i,1}$ und $v_{i,2}$, nacheinander betrachtet. Anschließend werden zwei Knoten v_3 und v_4 gesucht, die auf unterschiedlichen Seiten der Kante liegen. Dabei gilt: Das Tripel $(v_{i,1}, v_{i,2}, v_3)$ bildet einen Rechtsknick und das Tripel $(v_{i,1}, v_{i,2}, v_4)$ einen Linksknick. Danach muss überprüft werden, ob $v_{i,1}$ und $v_{i,2}$ auf verschiedenen Seiten der Kante liegen, die durch

3. Theoretischer Hintergrund

v_3 und v_4 gebildet wird. Sind alle diese Bedingungen erfüllt, so ist die Kante (v_3, v_4) eine Schnittkante von e_i . Der Vorteil dieses Algorithmus liegt darin, dass alle vier Knicküberprüfungen notwendige Eigenschaften darstellen. In der Praxis werden dadurch viele Knoten bereits ausgeschlossen, da sie die erste Knickbedingung nicht erfüllen.

Schnittkanten ausschließen

Um die Kanten zu finden bei denen die Schnittkanten zu viele Kanten ausschließen, wird jede Kante einzeln betrachtet.

MP: Kann man diesen Algorithmus nicht einfacher als Text erklären?

FA: Ist das Besser, also sollte ich probieren möglichst wenig als Algo zu machen?

Für jede Kante wird folgender Algorithmus aufgerufen:

Algorithm 1 EXCLUDE EDGE INTERSECTION(e)

```
1: erstelle Map node_count mit Key Knoten und Value Integer
2: for  $e_i \in I(e)$  do
3:   seien  $v_1$  und  $v_2$  die Knoten von  $e_i$ 
4:   node_count[ $v_1$ ] += 1
5:   node_count[ $v_2$ ] += 1
6: end for
7: for  $v \in \text{node\_count}$  do
8:   if  $\deg(v_1) - \text{node\_count}[v] > \deg^*(v_1)$  then
9:     return True
10:  end if
11: end for
12: return False
```

Kanten Theoretisch ausschließen

Damit Kanten ausgeschlossen werden können muss zunächst die Instanz mit einem beliebigen Lösungsansatz gelöst werden. Nun muss unterschieden werden wie viele Lösungen die Instanz hat. Sollte die Instanz nur eine Lösung haben so sind alle gefundenen Kanten fixierbar und alle anderen Kanten können ausgeschlossen werden. Sollte es mehr als eine Lösung geben muss nach jedem Lösen eine neue Beschränkung hinzugefügt werden die mindestens eine Kante der gefundenen verbietet. Dies wird so lange wiederholt bis keine Lösungen mehr gefunden werden. Nun können alle jemals gefundenen Kanten als fixierbar betrachten und alle anderen Kanten als ausschließbar.

OrTools und Gurobi

Da OrTools und Gurobi eine ähnliche Art der Formulierung verwenden werden sie hier zusammengefasst. Für die Überschneidungsbeschränkung wird die Funktion *add_at_most_one*

3.2. Kanten Formulierung

von OrTools verwendet. Diese Funktion wird mit den folgenden Variablenpaaren aufgerufen.

$$(x_{e_1}, x_{e_2}) \quad \forall e_1, e_2 \in E : e_2 \in I(e_1)$$

Gurobi nutzt hier äquivalent

$$x_{e_1} + x_{e_2} \leq 1 \quad \forall e_1, e_2 \in E : e_2 \in I(e_1)$$

Für die Gradbeschränkung wird die Summe der booleschen Variablen x_e für alle Kanten eines Knotens i auf den Sollgrad $\deg^*(i)$ beschränkt.

Dafür werden folgende Bedingungen hinzugefügt:

$$\sum x_{e_i} = \deg^*(i) \quad \forall i \in V$$

Wobei e_i alle Kanten für den Knoten i sind.

Für die *alternative Kantenbeschränkung* werden folgende Bedingungen hinzugefügt:

$$\sum_{j \in I(e)} x_j + x_e \leq 1 \quad \forall e \in E$$

Für die fixierung bzw. ausschließung der Kanten werden die zugehörigen Variablen auf *Wahr* oder *Falsch* gesetzt.

PySat

Da PySat eine SAT Formulierung also Klauseln in der KNF verwendet, müssen die Bedingungen etwas angepasst werden. Die Überschneidungen können sehr ähnlich mit folgender Formel umgesetzt werden:

$$\neg x_{e_1} \vee \neg x_{e_2} \quad \forall e_1, e_2 \in E : e_2 \in I(e_1)$$

Für die Gradbeschränkung ist das Problem das bei PySat für eine Menge von Variablen nicht festgelegt werden kann wie viele von ihnen wahr sein müssen. Hierfür gibt es zwei Ansätze. Der erste Ansatz nutzt die Kardinalitätscodierungen die PySat anbietet.

PK: hier aufpassen mit den Spaces

FA: @Phillip meinstest du das ich hier das tilde zeichen einfügen soll?

Hierfür kommen nur *sequential counters* [20], *sorting networks* [3], *cardinality networks* [1], *totalizer* [2], *modulo totalizer* [18], *modulo totalizer for k-cardinality* [16] und eine Native Codierung für manche Solver in Frage. Die Restlichen Codierungen bieten keine Möglichkeit um k-Kardinalitäten zu erzeugen. Also das k viele Variablen wahr sind. Bei diesem Ansatz gibt es noch die Möglichkeit das man die Kardinalität auf den exakten Sollgrad setzt, oder festlegt das der Knoten mindestens den Sollgrad hat. Insgesamt erzielen beide Bedingungen denselben Effekt da der Maximale Grad durch die Anzahl der Kanten begrenzt ist. Wenn jedoch nur gefordert wird, dass der Knoten mindestens den Sollgrad erreicht, müssen weniger Hilfsvariablen eingeführt werden. Bei der Exakten

3. Theoretischer Hintergrund

Methode könnte der Solver allerdings schneller zu einer Lösung finden da diese Bedingung stärker ist, also mehr Informationen liefert.

Die andere Methode ist die sogenannte *subset* Methode. Bei dieser Methode wird für jeden Knoten i eine Menge von Klauseln erstellt. Diese Klauseln umfassen alle Kombinationen der Kanten E_i der Größe

$$|E_i| - (\deg^*(i) - 1)$$

wobei E_i die Menge aller Kanten für den Knoten i ist. Angenommen, der Knoten v_1 besitzt die Kanten e_1, e_2, e_3, e_4 und den Sollgrad 2. Dann werden folgende Klauseln erstellt

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_4) \wedge (x_1 \vee x_3 \vee x_4) \wedge (x_2 \vee x_3 \vee x_4)$$

Damit diese vier Klauseln erfüllt sind, müssen mindestens zwei der Variablen wahr sein. Die Idee basiert darauf das für jede Klausel angenommen wird, dass nur noch eine Kante zum Erreichen des Sollgrads fehlt und die in der Klausel genannten Kanten noch nicht aktiv sind. Insgesamt wird so eine Kardinalitäts Codierung erzeugt. Diese Methode hat den Vorteil, dass keine Hilfsvariablen benötigt werden, allerdings entstehen sehr viele Klauseln.

FA: sollte ich hier noch nennen wie der *sequential counter* oder der Native Ansatz funktioniert weil diese am Ende am besten sind. Ich müsste dann aus den Experimenten vorgeifen. Auch brauche ich Information nicht wirklich da ich PySat hier als Blackbox nutzen kann.

PK: den Sequential Counter kannst Du hier gern erklären.

FA:
Ich glaube inzwischen das die native Codierung durch Glucard4 besser ist, soll ich hier nur erklären was ich am ende nutze obwohl ich dann aus den Experimenten vorgeife?

Die Fixierung von Kanten kann analog wie in OrTools und Gurobi umgesetzt werden. Hierbei werden anstatt die Variablen auf *Wahr* und *Falsch* zu setzen, die Klausen (x_e) und $(\neg x_e)$ hinzugefügt.

Für die *alternative Kantenbeschränkung* können folgende Klauseln genutzt werden:

$$x_e \vee \bigvee_{e_j \in I(e)} x_{e_j} \quad \forall e \in E$$

3.3. Dreieck Formulierung

Bei der Dreiecksformulierung wird nicht jeder Kante eine Variable zugeordnet. Stattdessen erhält jedes mögliche Dreieck eine eigene Variable. Die Überschneidungen können wie bei der Kantenformulierung übernommen werden, da die Variablen entsprechend angepasst wurden. Auch die Gradbeschränkung kann aus der Kantenformulierung übernommen werden. Lediglich die Knoten auf der Hülle müssen gesondert betrachtet werden. Für diese Knoten muss der Sollgrad um eins reduziert werden, da ein Dreieck am Rand beide

Kanten stellt. Auch bei der Dreiecksformulierung können Dreiecke ausgeschlossen werden. Dafür können die gleichen Bedingungen wie beim Ausschluss von Kanten verwendet werden. Es werden dann alle Dreiecke ausgeschlossen, die eine ausgeschlossene Kante enthalten.

3.3.1. Implementierung der Dreiecksformulierung

Die Implementierung der Dreiecke kann in weiten Teilen von der Implementierung der Kanten übernommen werden. Angepasst werden müssen lediglich die Berechnung der Dreiecke, deren Überschneidung sowie die Bestimmung aller Dreiecke für einen gegebenen Knoten.

Zur Erstellung aller Dreiecke werden zunächst alle möglichen Kanten hinzugefügt. Es werden also wieder alle Kanten betrachtet, die keinen Knoten schneiden. Anschließend werden alle Knoten nacheinander durchlaufen. Hierbei kann gleichzeitig zum Beispiel ein *Map* genutzt werden um für jeden Knoten die zugehörigen Dreiecke zu speichern. Für jeden Knoten werden alle Nachbarn bestimmt und mit den anderen Knoten diesen Nachbarn alle möglichen Zweierkombinationen gebildet. Wenn eine dieser Zweierkombinationen eine Kante besitzt, wird ein Dreieck erstellt. Für eine Triangulierung sind jedoch nur die Dreiecke relevant, die keine weiteren Knoten enthalten. Es werden daher nur die Dreiecke gespeichert, die keine zusätzlichen Knoten enthalten. Dies kann wieder mit *shapely* überprüft werden. Zusätzlich wird darauf geachtet das jedes Dreieck nur einmal gespeichert wird, da mit dieser Methode jedes Dreieck theoretisch bis zu drei Mal betrachtet werden kann.

Die Überschneidung wird mit der *Shapely* Bibliothek berechnet. Dafür werden alle Dreiecke paarweise verglichen. Dabei ist zu beachten das *Shapely* eine Überschneidung schon bei einer Berührung bestätigt. Es muss also zusätzlich getestet werden ob sie sich nicht Berühren.

FA: Sollte ich hier erwähnen, dass ich dies auch mit CGAL implementiert habe, es jedoch langsamer ist?

MP: Kannst du machen würde ich aber so nicht schreiben. Prinzipiell sollte CGAL schon schneller sein als Shapely wenn beide unter den gleichen Exaktheitsbedingungen arbeiten. Es kann sein, dass Shapely dir falsche Ergebnisse liefert, weil es mit Floats rechnet. Ich würde eher sagen du hast auch eine exakte Implementierung mit CGAL die aber aufgrund der Exaktheit langsamer ist und daher wurde in den späteren Experimenten Shapely genutzt.

FA: Ich kann mir gut vorstellen das meine Implementierung vorallem schlechter ist. Weil vorallem das mit dem Berühren in CGAL nerviger war. Sollte ich da irgendwas schreiben oder es einfach so lassen. Oder sollte ich genau das schreiben das es auch eine Implemntierung in CGAL gibt diese aber langsamer ist weil Berührungen schlechter von mir implementiert wurden?

Die Formulierungen für OrTools, Gurobi und PySat können analog zur Kantenformulierung umgesetzt werden.

3.4. Optimierungsproblem

Im Gegensatz zu den vorherigen Ansätzen wird das Problem in diesem Abschnitt nicht als Entscheidungsproblem, sondern als Optimierungsproblem betrachtet. Dies hat zum einen den Vorteil, dass bei einem *Timeout* des Solvers nicht der gesamte Fortschritt verloren geht. Ein weiterer Vorteil ist, dass bei Nein-Instanzen versucht wird, dennoch eine Lösung zu finden. Das bedeutet, sollte es in bestimmten Bereichen nicht möglich sein, eine gradbeschränkte Triangulierung zu finden, aber diese benötigt werden, kann zumindest eine möglichst nahe Triangulierung bestimmt werden.

Dafür wurde die Formel (3.3) eingeführt. Diese Formel bewertet wie weit die durchschnittliche Grad Abweichung von der Gradbeschränkung entfernt ist. Es werden drei Ansätze vorgestellt. Der erste ist ein naiver Ansatz der versucht mit Flips die Gradbeschränkung zu erreichen. Die anderen beiden Ansätze verwenden OrTools. Hierbei können die Bedingung aus der oben beschriebenen Kantenformulierung übernommen werden. Nur die Gradbeschränkung wird durch eine Zielfunktion ersetzt. Zum einen wird als Zielfunktion die Formel (3.3) genutzt. Der andere Ansatz versucht die maximale Gradabweichung zu minimieren.

Für die Bewertung der Lösungen wird folgende Formel verwendet. Diese liefert einen Durchschnitt über alle Prozentualen Gradabweichung.

$$\text{diff}_i = \left| \deg^*(i) - \deg(i) \right| \quad (3.1)$$

$$e_i = \begin{cases} \frac{\deg^*(i) - \text{diff}_i}{\deg^*(i)} & \deg^*(i) \geq \text{diff}_i \\ 0 & \text{sonst} \end{cases} \quad (3.2)$$

$$Q_T = \sum_{i=0}^n \frac{e_i}{n} \quad (3.3)$$

Die Formel liefert eine Bewertung (Q_T) für eine Triangulierung mit n Knoten. Hierbei gibt $\deg^*(i)$ den gewünschten Grad an der Stelle i an. Dazu passend gibt $\deg(i)$ den aktuellen Grad an. In der Berechnung (3.1) wird die Difference zwischen dem gewünschten und aktuellen Grad berechnet. In (3.2) wird diese Abweichung Prozentual bewertet. Am Ende wird in (3.3) ein Durchschnitt über alle Prozentualen Werte gebildet.

3.4.1. Flips

In dieser Heuristik werden die in Definition 2.6 definierten Kantenflips verwendet. Ziel ist es, die Gradbeschränkung zu erreichen. Dabei wird ausgenutzt, dass Flips den Grad eines Knotens verändern können die Triangulationseigenschaft dabei aber erhalten bleibt.

Das Verfahren beginnt mit einer initialen Triangulierung. Diese wird durch ein klassisches Triangulationsverfahren erzeugt. Anschließend werden maximal k Flips durchgeführt. Ziel ist es, die Gradbeschränkung zu erfüllen.

Zunächst wird mit dem Algorithmus 2 eine Kante ausgewählt. Diese soll geflippt werden. Der Algorithmus versucht, eine Kante zu finden, deren Knoten nicht den Sollgrad erfüllen.

Zusätzlich wird über den Parameter p eine Wahrscheinlichkeit eingeführt. Dadurch kann auch eine Kante ausgewählt werden, deren Knoten den Sollgrad erfüllen. Dies verhindert, dass das Verfahren in einem lokalen Optimum stecken bleibt.

Die ausgewählte Kante wird anschließend wie folgt geflippt. Für jeden Flip wird zufällig eine Kante e_0 ausgewählt. Für diese Kante werden zunächst alle Dreiecke gesucht, die diese Kante enthalten. Dies geschieht, indem für die beiden Knoten $x_{0,0}$ und $x_{0,1}$ der Kante e_0 alle ausgehenden Kanten verglichen werden. Sollten zwei dieser Kanten den gleichen Knoten v haben, muss das Tripel $(x_{0,0}, x_{0,1}, v)$ ein Dreieck bilden, welches die Kante e_0 enthält. Sollten mehr als zwei Dreiecke gefunden werden, so müssen die beiden Dreiecke ausgewählt werden, welche leer sind (keine weiteren Knoten enthalten). Da es in einer Triangulierung nicht zu Überschneidungen kommen darf, kann es nur zwei leere Dreiecke geben. Aus diesen beiden Dreiecken $(x_{0,0}, x_{0,1}, v_0)$ und $(x_{0,0}, x_{0,1}, v_1)$ wird dann die Kante e_0 entfernt und durch die neue Kante, bestehend aus v_0 und v_1 , ersetzt.

Nach jedem Flip wird überprüft, ob die Gradbeschränkung für alle Knoten erfüllt ist. Sollte dies der Fall sein, wird das Verfahren beendet.

Algorithm 2 CHOOSE EDGE(v, p)

```

1:  $E \leftarrow \{e \mid e \text{ ist ausgehende Kante von } v\}$ 
2: while  $E \neq \emptyset$  do
3:    $e \leftarrow$  zufällige Kante aus  $E$ 
4:    $E \leftarrow E \setminus \{e\}$ 
5:    $u \leftarrow$  Knoten von  $e$  and  $v \neq u$ 
6:   if  $\deg(u) \neq \deg^*(u)$  then
7:     return  $e$ 
8:   else if zu  $p$  % then
9:     return  $e$ 
10:  end if
11: end while
12: return  $e$ 

```

3.4.2. OrTools mit Durchschnitts Abweichung

Für die erste Zielfunktion wird sich an der Gleichung (3.3) orientiert. Hier wird die Zielfunktion definiert durch die Maximierung von Q_T . Die Teiler in (3.3) und (3.2) werden entfernt. Diese sind für die Maximierung nicht relevant. Da OrTools keine Betragsfunktion in Formeln erlaubt, muss die Funktion *AddAbsEquality* verwendet werden. Diese erzeugt den Betrag mit den folgenden Bedingung

$$\begin{aligned}
abs_i &\geq \deg^*(i) - \deg(i) \\
abs_i &\geq \deg(i) - \deg^*(i) \\
abs_i &= (\deg^*(i) - \deg(i)) \vee (\deg(i) - \deg^*(i))
\end{aligned}$$

Die Variable abs_i ist hierbei eine Integer-Hilfsvariable, die den Betrag der Differenz erhält. Diese Variable kann dann für die Formel (3.1) genutzt werden.

3. Theoretischer Hintergrund

Eine Fallunterscheidung für (3.2) ist ebenfalls nicht möglich. Die Variablen e_i werden daher mit folgender Formel berechnet.

$$e_i = \deg^*(i) - \min(\text{diff}_i, \deg^*(i))$$

OrTools erlaubt allerdings keine *min*-Funktion in Formeln. Deshalb wird die Funktion *AddMinEquality* verwendet. Diese nutzt das eine Minimierung mit Folgender Formel in ein Maximierung umgewandelt werden kann.

$$\min(a, b) = -\max(-a, -b)$$

Eine Betragsfunktion kann auf eine Maximierungsfunktion reduziert werden. Da dies im obigen Beispiel passiert ist, können hier die gleichen angepassten Bedingungen genutzt werden.

zusammengefasst bekommen wir folgende Bedingungen

$$\begin{aligned} abs_i &\geq \deg^*(i) - \deg(i) \\ abs_i &\geq \deg(i) - \deg^*(i) \\ abs_i &= (\deg^*(i) - \deg(i)) \vee (\deg(i) - \deg^*(i)) \\ min_i &\geq -abs_i \\ min_i &\geq -\deg^*(i) \\ min_i &= (-abs_i) \vee (-\deg^*(i)) \\ e_i &= \deg^*(i) - (-min_i) \\ Q_T &= \sum e_i \end{aligned}$$

Als Zielfunktion wird dann Q_T maximiert.

3.4.3. OrTools mit Minimierter Maximaler Abweichung

Als zweite Zielfunktion wird für OrTools die maximale Abweichung minimiert. Dafür wird eine Integer-Hilfsvariable *max_min* eingeführt. Zusätzlich wird die *abs_i* Variable mit ihren Bedingungen aus Abschnitt 3.4.2 verwendet. Anschließend werden folgende Bedingungen eingeführt.

$$\begin{aligned} max_min &\geq abs_i && \forall i \in V \\ max_min &\geq 0 \end{aligned}$$

Die Variable *max_min* wird anschließend minimiert.

3.5. Conflict-driven clause learning

In diesem Kapitel werden Sat-Solver nochmal genauer betrachtet. Dabei werden nicht weitere Beschränkungen hinzugefügt sondern der Solver wird während des Lösens beeinflusst.

3.5. Conflict-driven clause learning

Dafür wird *Conflict-driven clause learning* (CDCL) betrachtet. Die in diesem Kapitel vorgestellten Informationen stammen aus dem Buch *The Art of Computer Programming, Volume 4, Fascicle 6: Satisfiability* [14]. Im Allgemeinen funktionieren SAT-Solver so, dass sie jede Möglichkeit ausprobieren. Dies lässt sich als Baum darstellen (siehe Abbildung 3.5). Das Besondere am CDCL-Ansatz ist, dass nicht die Äste iterativ abgearbeitet

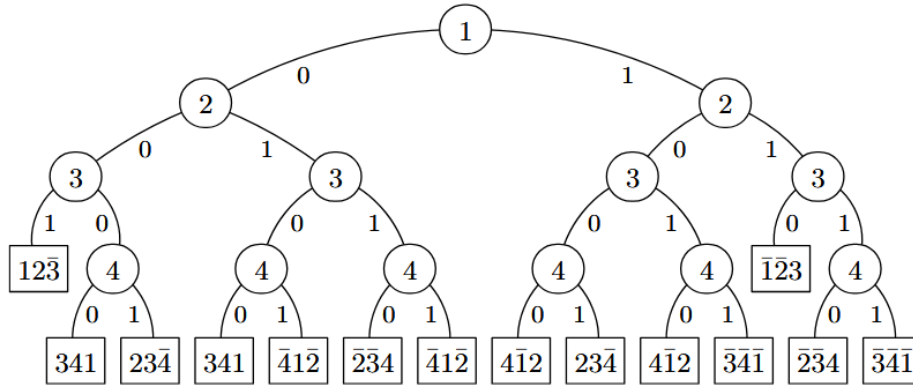


Abbildung 3.5.: Ein Baum welcher die Möglichen Entscheidungen eines SAT-Solvers darstellt. Jeder Ast entspricht Belegungen der Literale

werden, sondern dass es einen Pfad gibt. Der Pfad ist eine aktuelle Teilbelegung der Literale. Theoretisch entspricht dieser einem Ast im Baum, aber die Auswahl der Belegung basiert nicht direkt auf dem Baum. Zusätzlich muss für jedes belegte Literal eine Begründung angegeben werden. Diese Begründungen werden bei Konflikten relevant.

3.5.1. Unit-Klauseln

Der Grundansatz zur Erweiterung des Pfades besteht darin, *Unit-Klauseln* zu finden. Das sind Klauseln, in denen nur ein Literal noch nicht belegt ist und die Klausel nicht erfüllt ist. Zum Beispiel kann durch die Klausel $(\bar{x}_1 \vee x_2 \vee \bar{x}_3)$ und den Pfad $(\bar{x}_2, x_3)$ das Literal $\bar{x}_1$ dem Pfad hinzugefügt werden. Die Klausel dient dabei als Begründung.

Sollten keine weiteren *Unit-Klauseln* existieren, wird basierend auf Heuristiken ein Literal belegt. Bei dieser Art der Belegung wird intern vermerkt, dass eine neue Entscheidungsebene erreicht wurde. Die Entscheidungsebenen werden auch bei Konflikten wichtig. Es ist zu beachten, dass jeder Abschnitt des Pfades einer Entscheidungsebene zugeordnet werden kann. Ein Literal, das weiter rechts im Pfad steht, wird einer höheren Entscheidungsebene zugeordnet.

3.5.2. Konflikte

Ein Konflikt tritt auf, wenn ein Literal zum zweiten Mal dem Pfad hinzugefügt werden soll, das Literal aber bereits eine andere Belegung besitzt. Hier zeigt sich der besondere Ansatz von CDCL. Kommt es zu einem Konflikt, wird nicht einfach zu einer vorherigen

3. Theoretischer Hintergrund

Entscheidungsebene zurückgekehrt. Stattdessen werden die Informationen, wie es zu dem Konflikt kam, genutzt. Sei

$$c = (\bar{l} \vee \bar{a}_1 \vee \dots \vee \bar{a}_k)$$

eine Klausel, die einen Konflikt aufweist. Das bedeutet, alle Literale wurden dem Pfad hinzugefügt und die Klausel ist nicht erfüllt.

Es kann angenommen werden, dass l und ein weiteres Literal a_i aus c auf der höchsten Entscheidungsebene d liegen. Andernfalls wäre c eine *Unit-Klausel* und somit wäre l bereits auf $d - 1$ belegt worden. Weiterhin kann angenommen werden, dass das Literal l das rechteste Literal von allen Literalen aus c im Pfad ist. Also das Literal aus c , das zuletzt belegt wurde. Daraus folgt, dass l keine neue Entscheidungsebene erzeugt hat, da zumindest a_i vor l auf der Entscheidungsebene d belegt wurde.

Es existiert somit eine Begründung

$$(\bar{l} \vee \bar{r}_1 \vee \dots \vee \bar{r}_k)$$

für die Belegung von l . Damit kann folgende Klausel erzeugt werden

$$c' = (\bar{a}_1 \vee \dots \vee \bar{a}_k \vee \bar{r}_1 \vee \dots \vee \bar{r}_k)$$

obei c' mindestens eine Belegung aus der Entscheidungsebene d enthält. Sollte es ein weiteres Literal außer a_i geben, das auf der Entscheidungsebene d belegt wurde, ist c' ebenfalls eine Konfliktklausel. In diesem Fall wird c' rekursiv so lange erzeugt bis es genau ein Literal $\bar{x}$ gibt, das auf Entscheidungsebene d belegt wurde. Der rekursive Aufruf fügt dabei die Begründung des zweiten Literals aus Entscheidungsebene d ohne dieses Literal zu c' hinzu. Von den restlichen Literalen in c' wird dann die höchste Entscheidungsebene bestimmt und zu dieser zurückgekehrt. Anschließend wird $\bar{x}$ dem aktuellen Pfad hinzugefügt und c' als Begründung für die Belegung verwendet.

3.6. Instanzgenerierung

In diesem Kapitel werden die verschiedenen Instanzen Typen und ihre Generierung dargestellt. Auf diesen Instanzen werden die späteren Experimente ausgeführt.

Die Knotenmengen werden hierbei durch die von Python bereit gestellte Funktion *randint* geniert. Mit dieser werden so lange verschiedene Koordinaten berechnet bis eine Gewünschte Anzahl erreicht wurde.

3.6.1. Ja - Instanzen

Bei den Ja-Instanzen handelt es sich um Knotenmengen, bei denen die $\deg^*$ -Funktion Grade liefert, welche eine gradbeschränkte Triangulierung ermöglichen. Hierbei ist nicht bekannt, wie viele mögliche Lösungen existieren, sondern nur, dass es mindestens eine gibt.

Der generelle Ablauf zur Erzeugung von Ja-Instanzen beginnt mit einer zufälligen Knotenmenge. Dabei wird lediglich die Anzahl der Knoten vorgegeben. Anschließend

wird mit einem klassischen Triangulationsverfahren eine initiale Triangulierung erstellt. Von dieser werden dann die Grade gespeichert, um sie als Eingabe für die zu testenden Algorithmen zu verwenden.

Alle fünf Verfahren zur Erzeugung von Ja-Instanzen werden in Abbildung 3.6 visualisiert.

Delaunay

Das erste klassische Verfahren zur Gradbestimmung verwendet die Delaunay-Triangulierung. Die Delaunay-Triangulierung wurde bereits im Kapitel Abschnitt 1.3.1 vorgestellt. Für die Erstellung der Delaunay-Triangulierung wird die Bibliothek *scipy* genutzt.

Delaunay mit zufälligen Flips

Das zweite Verfahren zur Erzeugung von Ja-Instanzen basiert auf der Delaunay-Triangulierung. zusätzlich werden jedoch zufällige Flips durchgeführt. Zunächst wird eine Delaunay-Triangulierung der Knotenmenge erstellt. Auf dieser Triangulierung werden anschließend k zufällige Flips gemäß Definition 2.6 durchgeführt. Das genau verfahren wurde in Abschnitt 3.4.1 schon beschrieben. Dieser Prozess verändert die lokale Struktur der Triangulierung und erzeugt so Knotenmengen mit unterschiedlichen Gradverteilungen, behält aber die Triangulationseigenschaft bei.

Zufällige Kantenerzeugung

Das nächste Verfahren basiert auf der zufälligen Auswahl von Kanten. Hierbei werden zunächst alle geometrisch möglichen Kanten zwischen den Punkten identifiziert. In diesem Schritt werden Kanten ausgeschlossen, welche andere Knoten schneiden würden.

Die verbleibenden gültigen Kanten werden dann in einer zufälligen Reihenfolge betrachtet. Für die betrachteten Kanten wird geprüft ob sie eine der bereits ausgewählten Kanten schneidet. Nur wenn keine Schnittkanten entstehen, wird die Kante der Triangulierung hinzugefügt.

Dieses Verfahren ist maximal da alle möglichen Kanten betrachtet wurden.

Iteration

Ein weiteres Verfahren zur Erzeugung von Ja-Instanzen basiert auf der Iteration. Hierbei werden zunächst alle möglichen Kanten zwischen den Punkten identifiziert. Diese werden nun nacheinander betrachtet und wenn sie keine andere Kante schneiden hinzugefügt. Diese Verfahren ist ebenfalls maximal, da alle möglichen Kanten betrachtet wurden. Beim Iterativen Verfahren ist zu beachten das Triangulationen erzeugt werden welche einen Knoten haben welcher einen sehr hohen Sollgrad hat. Dies basiert darauf das die Kanten in der Reinform betrachtet werden wie sie erstellt werden. Das hat den Effekt das alle Kanten des ersten Knoten hinzugefügt werden können da es noch keine anderen Kanten gibt welche diese schneiden könnten.

3. Theoretischer Hintergrund

Greedy

Das letzte Verfahren ist das Greedy Verfahren. Hierbei werden auch alle möglichen Kanten zwischen den Punkten identifiziert. Allerdings werden sie im Gegensatz zum Iterativen Verfahren zunächst nach der Länge aufsteigend sortiert. Anschließend wird wie beim Iterativen Verfahren jede Mögliche Kante nacheinander hinzugefügt.

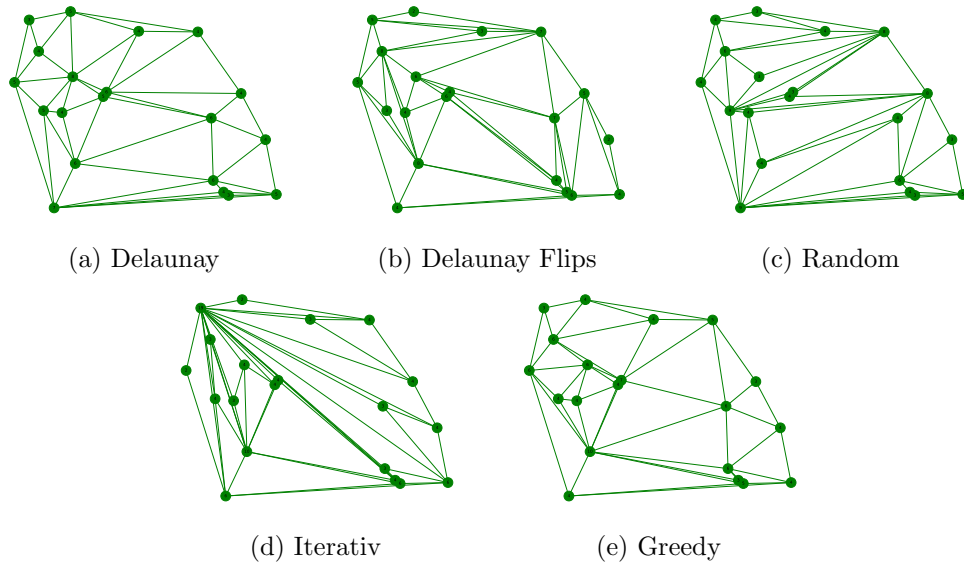


Abbildung 3.6.: Visualisierung der fünf verschiedenen Instanztypen.

3.6.2. Nein-Instanzen

Nein-Instanzen sind Knotenmengen, bei denen die $\deg^*$ Funktion Gradwerte liefert, die keine gradbeschränkte Triangulierung zulassen. Diese Instanzen entstehen nicht direkt, sondern werden aus Ja Instanzen abgeleitet. Nach der Generierung muss zusätzlich geprüft werden, ob die Instanz nicht möglich ist. Da die beschriebenen Verfahren nur mit hoher Wahrscheinlichkeit Nein Instanzen erzeugen. Es ist zu beachten, dass die Summe der Sollgrade mithilfe der Polyederformel einfach berechnet und überprüft werden kann. Daher besitzen die abgeleiteten Nein Instanzen insgesamt die gleichen Sollgrade wie die Ja-Instanzen. Zusätzlich darf kein Sollgrad größer als die Anzahl der Knoten abzüglich eins sein, da sonst der Sollgrad nicht erfüllt werden kann da nicht genügend Kanten erzeugt werden können.

Bei der ersten Methode werden zwei Knoten ausgewählt. Vom ersten Knoten wird ein Wert abgezogen und beim zweiten Knoten derselbe Wert hinzugefügt. Zunächst wird eine zufällige Zahl r zwischen null und c gewählt. Hierbei ist c eine konstante Zahl, die beim Generieren festgelegt wird. Anschließend werden zwei Knoten v_1 und v_2 ausgewählt.

Dabei müssen folgende Bedingungen erfüllt sein

$$v_1 \neq v_2 \quad (3.4)$$

$$\deg^*(v_1) < r + 2 \quad (3.5)$$

$$\deg^*(v_2) + r > n - 1 \quad (3.6)$$

wobei n die Anzahl der Knoten ist. Wenn beide Knoten die Bedingungen erfüllen, wird der Sollgrad von v_1 um r verringert. Der Sollgrad von v_2 wird um r erhöht.

Die zweite Methode besteht darin, zwei Knoten auszuwählen und deren Sollgrade zu tauschen. Es muss lediglich beachtet werden, dass es sich um zwei verschiedene Knoten handelt.

Beide Ansätze werden mehrfach angewendet, um die Wahrscheinlichkeit zu erhöhen, dass die Instanzen nicht mehr möglich sind. Knoten werden dabei allerdings nicht mehrfach betrachtet.

Die erste Methode wird im folgenden *move* genannte. Die andere Methode wird *change* genannt.

FA: Ist es sinnvoll hier Namen zu verwenden, und sollte ich diese ins Deutsche übersetzen?

3.7. Theoretische Mehrere Lösungen

MP: Gibt es hierzu noch zitierbare Literatur?

FA: Ich habe zur gradbeschränkten Triangulierungen an sich schon keine Literatur gefunden ich wüsste nicht wo ich dazu Lösungen finden könnte.

FA: @Michael Als wir darüber gerdet hatten hast du so eine ähnliche Instanz an die Tabel und gesagt die ist von irgendwem. Darauf basierend hab ich die hier gemacht. Muss ich da irgendwie zitieren?

Bei Ja-Instanzen stellt sich die Frage, ob sie eindeutig sind. Das bedeutet, ob es pro Ja-Instanz nur eine Lösung gibt und ob die Instanz nicht mehr erfüllbar ist, sobald eine Kante dieser Lösung verboten wird. Die Antwort darauf ist nein, es existieren Instanzen, bei denen mehrere Lösungen möglich sind. Wie in Abbildung 3.7 zu erkennen ist, existieren zwei verschiedene Lösungen für die gleiche Ja-Instanz. Die Zahlen an den Knoten geben hierbei den Sollgrad an. Diese Eigenschaft wird auch nochmal in Abschnitt 4.12.1 im Zusammenhang mit der Schwierigkeit einer Instanz untersucht. Das es mehrere Lösungen gibt lässt sich darauf zurück führen dass der Sollgrad hier *Im* und *Gegen* den Uhrzeigersinn erfüllt wird. Diese Art von Konstruktion kann theoretisch unendlich groß fortgesetzt werden, diese Eigenschaft könnte theoretisch genutzt werden auf andere Probleme zu reduzieren. Also die Anzahl der Lösungen einer Instanz kann als eine Eigenschaft dieser betrachtet werden.

3. Theoretischer Hintergrund

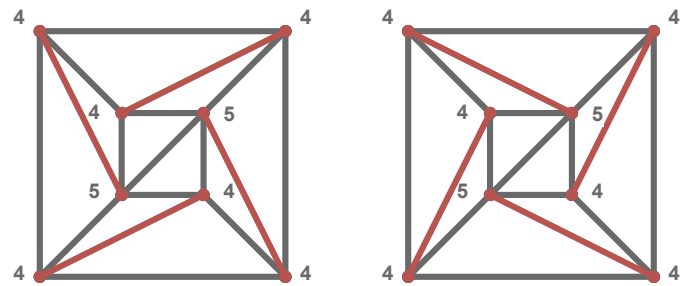


Abbildung 3.7.: Eine Ja-Instanz mit zwei möglichen Lösungen. Die Zahlen an den Knoten geben hierbei den Sollgrad an. Die roten kanten sind jene die in den Lösungen unterschiedlich sind.

4. Auswertung

In diesem Kapitel werden die Ansätze aus Kapitel 3 ausgewertet. Die ersten beiden Experimente testen, ob die Solver bei gleichen Bedingungen die gleichen Ergebnisse liefern. Die beiden Experimente danach werden genutzt, um den besten SAT-Solver für das gradbeschränkte Triangulierungsproblem aus Abschnitt 3.2.1 zu finden. Danach folgt in Abschnitt 4.6 ein Experiment, welches die verschiedenen Solver mit den verschiedenen Beschränkungen vergleicht, um den besten Solver zu bestimmen. Dieser wird genutzt, um in den folgenden Experimenten zum einen die theoretische Ausschließung von Kanten zu testen, Ja-Instanzen und Nein-Instanzen getrennt voneinander zu betrachten und möglichst große Instanzen zu lösen. Anschließend werden in Abschnitt 4.10 die Zielfunktionen betrachtet. In dem darauf folgenden Experiment wird der CDCL-Ansatz aus Abschnitt 3.5 genauer betrachtet, um während des Lösens Einfluss auf den Solver zu nehmen. Zum Abschluss wird in drei verschiedenen Experimenten versucht, die Schwierigkeit der Instanzen zu bestimmen.

4.1. Versuchsumgebung

PK: Ist sowas wie Pandas relevant? Das wird doch nur in der Auswertung gebraucht...

FA: Dann ist Slurminade und Algbench doch genauso irrelevant oder?, oder das die Plots mit plt und Seaborn erstellt wurden?

- Algbench und Slurminade, Pandas
- plt und Seaborn

In diesem Abschnitt werden die Bedingungen für die Experimente beschrieben. Alle Experimente welche eine Laufzeit beinhalten wurden entweder auf *AMD Ryzen 7 5800X @ 8x 3.8GHz-4.7GHz* mit *128GB* Arbeitsspeicher oder auf *AMD Ryzen 9 7900 @ 12x 3.7GHz-5.4GHz* mit *96GB* Arbeitsspeicher durchgeführt. Im folgenden werden in den Experiment die *Ryzen* zwischen 7 und 9 unterschieden. Dabei wurden für alle Experimente mit Ausnahme von Abschnitt 4.9 ein Timeout von 5 Minuten (300 Sekunden) gesetzt. Alle Laufzeiten sind dabei die Vorbereitungszeit plus die Laufzeit des Solvers. Diese komplette Laufzeit musste in den 5 Minuten liegen.

Für die Instanzgenerierung wurden die in Abschnitt 3.6 beschriebenen Methoden verwendet. Es wurden Instanzen mit 30 bis 120 Knoten in 10er Schritten generiert. Wobei immer zwei Ja-Instanzen und zwei Nein-Instanzen pro Knotengröße generiert wurden.

4. Auswertung

Bei den Nein-Instanzen wurde jeweils eine *move* und eine *change* Instanz generiert. Im folgenden werden alle Instanzen nach der Knotengröße aufsteigend sortiert.

Alle Experimente wurden in Python 3.13.5 durchgeführt. Der Algorithmus zum finden von Überschneidungen wurde in c++ implementiert und mit *pybind11* in Python eingebunden. Ebenso wurde der CDCL-Ansatz in c++ implementiert.

4.2. Permutation

In diesem Experiment wird untersucht, ob es einen Einfluss hat, in welcher Reihenfolge die Knoten den Solvern übergeben werden. Es wurden Instanzen mit 60 bis 90 Knoten betrachtet und auf dem Ryzen 7 getestet. Als Beschränkungen kamen die Überschneidungsbeschränkung und die Gradbeschränkung zum Einsatz. Bei allen Solvern war das Ergebnis das die Laufzeiten bei den verschiedenen Permutationen unterschiedlich sind. Daraus folgt, dass die Permutation der Knotenmenge einen Einfluss auf die Laufzeit hat. In den folgenden Experimenten wird für jeden Lauf die Knotenmenge fünfmal in unterschiedlichen Permutationen übergeben und der Durchschnitt berechnet.

4.3. Mehrmalige Durchläufe

In diesem Experiment wird untersucht, ob sich bei mehrfachen Durchläufen derselben Instanz mit demselben Solver und gleichen Parametern sich die Laufzeit verändert. Auch hier wurden Instanzen mit 60 bis 90 Knoten betrachtet und auf dem Ryzen 7 getestet. Als Beschränkungen kamen die Überschneidungsbeschränkung und die Gradbeschränkung zum Einsatz. Gurobi und SAT hatten bei allen Instanzen die gleichen Laufzeiten. Bei OrTools hingegen variieren die Laufzeiten bei den meisten Durchläufen. Da aufgrund von Abschnitt 4.2 bereits mehrere Durchläufe mit unterschiedlichen Permutationen durchgeführt werden, werden die OrTools-Durchläufe nicht zusätzlich mehrfach ausgeführt.

4.4. SAT Gradbeschränkung

In diesem Experiment werden Verschiedene Methoden der Gradbeschränkung in einem SAT Solver verglichen. Hierbei wurden für alle drei Methoden, also *subset*, Kardinalität mit exaktem Sollgrad und Kardinalität mit Mindest Sollgrad, die Laufzeiten gemessen. Bei den Kardinalitäts Codierungen von PySat wurden auch alle möglichen Codierungen Getestet. Das Ziel ist es die Gradbeschränkung zu finden mit der besten Laufzeit, welche in den Folgenden Experimenten verwendet wird. Für die Knotengröße wurden Instanzen der Größe 30 bis 90 verwendet. Bei der *subset* Methode wurde nur die Instanz mit 30 Knoten verwendet. Da bei größeren Knotenmengen der Arbeitsspeicher zu klein war. Als zusätzliche Bedingung wurde die Überschneidungsbeschränkung verwendet. Für die Exakte Methode sieht man in Abbildung A.1 das *sequential counters* die besten Ergebnisse erzielen. Die einzige andere relevante Codierung ist *totalizer*, aber auch dieser ist vorallem bei Delaunay Flips unter *sequential counters*. Auch für den Mindest Sollgrad

ist *sequential counters* die beste Variante gefolgt von *totalizer* wie man in Abbildung A.2 sieht.

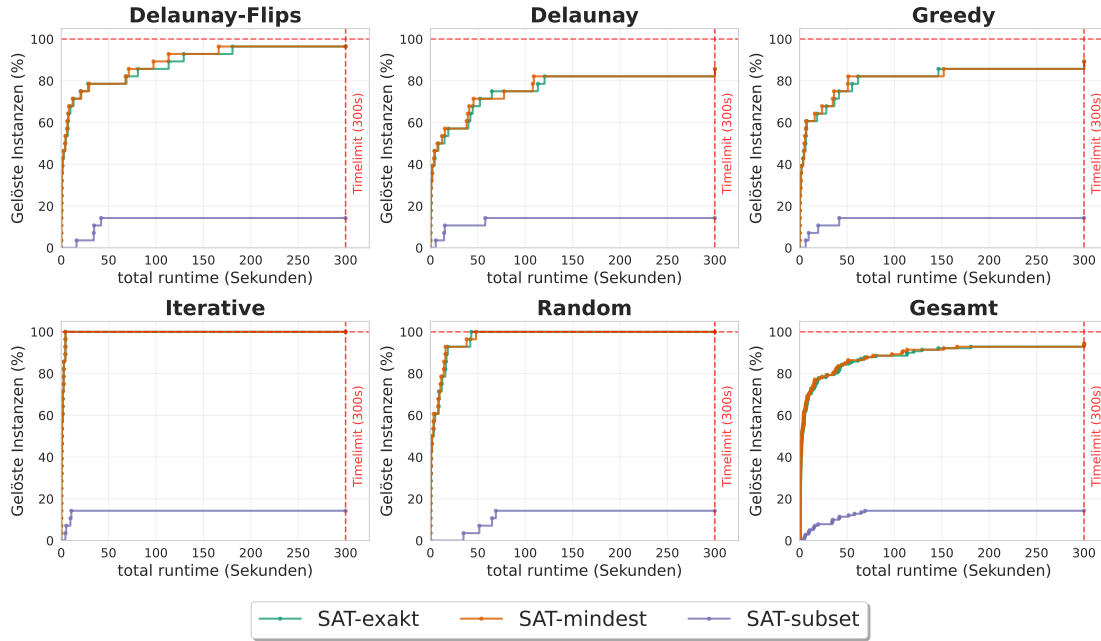


Abbildung 4.1.: In der Abbildung werden für alle Instanzen Arten die drei Kardinalitätsbeschränkungen verglichen. Die *subset* Methode ist die schlechteste, da sie nur für kleine Instanzen funktioniert. Die anderen beiden Methoden unterscheiden sich nicht signifikant.

Der Vergleich der drei Methoden ist in Abbildung 4.1 zu sehen. Hier wurden aus jedem der drei Ansätze das beste Ergebnis genommen. Hier ist klar erkennbar das *subset* die schlechteste Methode ist. Das lässt sich leicht daran erklären das nur die kleinsten Instanzgrößen gelöst werden konnten. Die anderen beiden Methoden unterscheiden sich nicht signifikant. Es ist zu sehen das öfter die Methode mit dem exakten Sollgrad langsamer ist. Dementsprechend werden in folgenden Experimenten die Kardinalitätsbeschränkung mit Mindest Sollgrad und *sequential counters* Codierung verwendet.

4.5. SAT Solver Vergleich

In diesem Experiment werden die verschiedenen SAT Solver verglichen, die von PySat bereitgestellt werden. PySat stellt *Cadical195*, *Gluecard4*, *Glucose42*, *Lingeling*, *MapleChrono*, *MapleCM*, *Maplesat*, *Mergesat3*, *Minicard* und *Minisat22* zur Verfügung. Von diesen wurden jeweils die neuesten verfügbaren Versionen verwendet. Alle Solver nutzen die *sequential counters* Codierung sowie die Überschneidungsbeschränkung. Bei *Gluecard4* und *Minicard* kam anstelle von *sequential counters* eine native Codierung der Gradbeschränkung zum Einsatz. Diese wird nur für diese beiden Solver angeboten und bietet eine

4. Auswertung

direkte Codierung ohne zusätzliche Variablen. Der *Lingeling* Solver wurde in großen Experimenten nicht berücksichtigt da er kein Timeout unterstützt. Er wurde aber zusätzlich mit dem besten Solver verglichen und hat schlechtere Ergebnisse erzielt. *CryptoMinisat* wurde nicht verwendet, da dieser Solver speziell für XOR-Probleme optimiert ist. Für die Instanzengröße wurden 60 bis 90 Knoten gewählt und es wurde auf dem Ryzen 9 getestet. Das Ziel besteht darin, den Solver zu identifizieren, der die besten Ergebnisse liefert, um diesen in zukünftigen Experimenten einzusetzen. In Abbildung 4.2 ist zu erkennen, dass

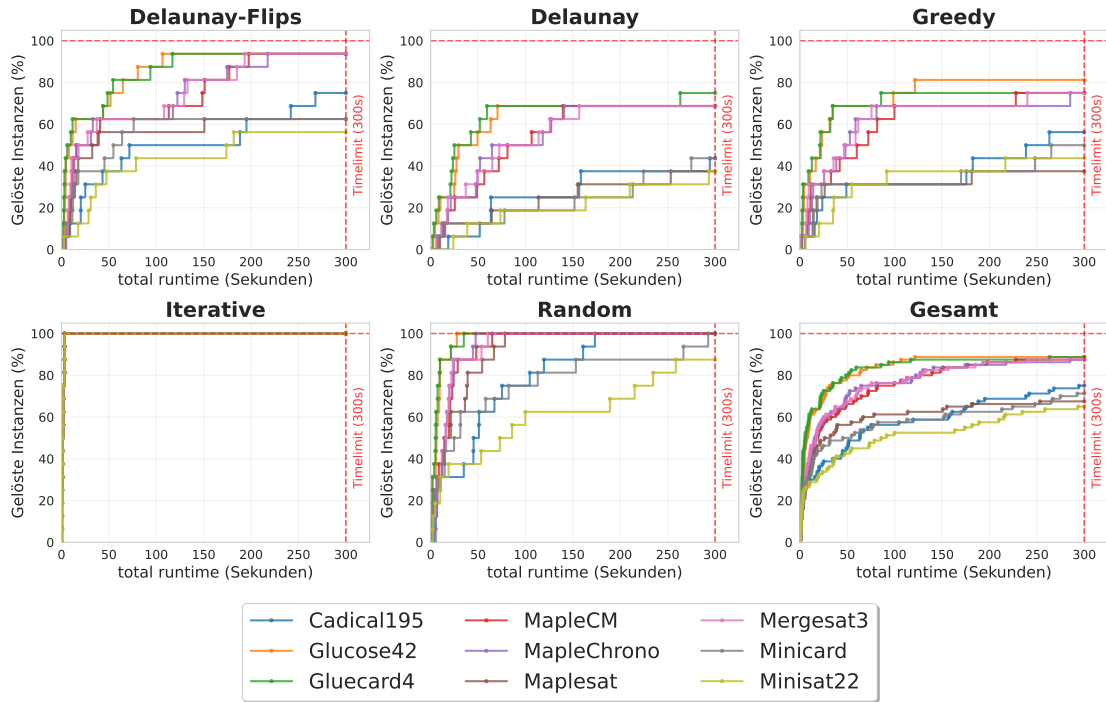


Abbildung 4.2.: Eval3 Gesamt

Gluecard4 und *Glucose42* die besten Ergebnisse erzielen. Da diese beiden Solver ähnliche Ergebnisse liefern ist nicht verwunderlich, da sie beide auf dem Glucose-Algorithmus basieren. Es zeigt sich aber dass bei diesem Problem weder der native noch der *sequential counters* Ansatz einen signifikanten Unterschied machen. Für den weiteren Verlauf wird *Glucose42* verwendet, da dieser bei Greedy eine Instanz in guter Zeit geschafft hat welche *Gluecard4* nicht geschafft hat.

4.6. Grund Solver

In diesem Experiment werden alle möglichen Bedingungen an die Solver übergeben und einzeln getestet. Untersucht werden die Solver SAT, OrTools und Gurobi jeweils mit der Kantenformulierung sowie der Dreiecksformulierung. Für alle Durchläufe wurden die Überschneidungsbeschränkung und die Gradbeschränkung genutzt. Bei der Kanten-

formulierung wurden das Fixieren und Ausschließen von Kanten sowie die *alternative Kantenformulierung* getestet. Bei der Dreiecksformulierung wurde nur das Ausschließen von Kanten getestet. Die Instanzengröße liegt bei 30 bis 120 Knoten und es wurde auf dem Ryzen 7 getestet.

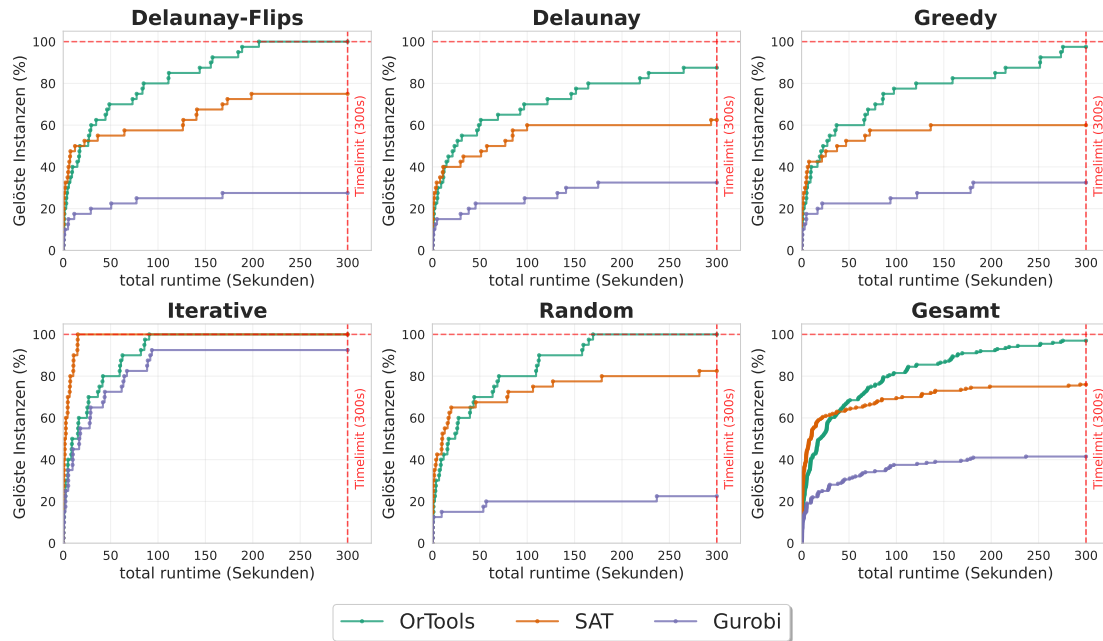


Abbildung 4.3.: Eval1 Gesamt

In Abbildung B.4 ist zu erkennen, dass Gurobi bis auf die Iterativen Instanzen keine guten Ergebnisse erzielt. Es wurden fast nie 50 Prozent der Instanzen gelöst. Es ist erkennbar das bei Gurobi bei kleinen Instanzen die alternative Kantenformulierung bessere Ergebnisse erzielt. Bei größeren Instanzen sind die standart Bedingungen besser.

SAT liefert die besten Ergebnisse, wie in Abbildung B.2 zu sehen ist beim fixieren der Hülle. Alle anderen Bediungen sind schlechter als nur die Überschneidungs und Gradbeschränkung. Es ist wieder zu sehen das Iterative warscheinlich zu einfach ist da der SAT solver hier alle Knotengrößen in kürzester Zeit schafft. Nur das ausschließen der Kanten schafft die größeren Instanzen nicht. Warscheinlich dauert hier das Ausschließen der Kanten zu lange.

In der letzten Abbildung B.3 ist zu sehen das OrTools mit der alternativen Kantenformulierung die besten Ergebnisse erzielt. Dies auch mit klarem Abstand wie in der Gesamt übersicht zu sehen ist. Es zeigt sich auch hier das das fixieren der Hülle etwas besser als die Standart Bedingungen ist.

In Abbildung 4.3 sind von allen Solvern die besten Bedingungen zu sehen. Hier ist zu sehen das OrTools die besten Ergebnisse erzielt. Gefolgt von SAT welches sogar bei kleinen Instanzen bessere Ergebnisse als OrTools erzielt. Gurobi ist in allen Fällen mit Abstand schlechter als SAT und OrTools. Es ist auch klar zu erkennen das die Dreiecksformulierung

4. Auswertung

bei allen Solvern schlechtere Ergebnisse erzielt als die Kantenformulierung. Im folgenden wird nur noch OrTools mit der alternativen Kantenformulierung und dem fixieren der Hülle betrachtet.

FA: Dreiecke sind noch nicht final da gen3 blockiert ist

FA: Kanten fixieren hatte einene Fehler und muss neu getestet werden. Sollte aber keine Relevanz haben und am Text nichts ändern.

4.7. Kanten vorab berechnen

In diesem Experiment wird die theoretische Ausschließung und Fixierung von Kanten untersucht. Für eine Auswahl von Instanzen wird vor dem Experiment eine Menge von Kanten bestimmt, die sicher ausgeschlossen oder fixiert werden können. Dies basiert auf der theoretischen Beschränkung aus Abschnitt 3.2. Die Grundlage für dieses Experiment ist die Annahme, dass das Ausschließen und Fixieren von Kanten in Abschnitt 3.2 nur schlecht performed hat, weil zu wenige Kanten ausgeschlossen wurden. In diesem Experiment wurde ausschließlich der SAT Solver verwendet, da SAT und OrTools in Abbildung 4.3 vergleichbare Ergebnisse liefern und dadurch die Übersichtlichkeit gewahrt bleibt. Als weitere Beschränkungen kamen nur die Überschneidungsbeschränkung und die Gradbeschränkung zum Einsatz. Für die Knotengröße wurden Instanzen mit 80 bis 120 Knoten gewählt.

FA: Finale Bilder analysieren

4.8. Ja/Nein getrennt

In diesem Experiment wird untersucht, ob der SAT Solver bei Ja-Instanzen oder bei Nein-Instanzen schneller ist. Es wird nur der SAT Solver mit den besten Parametern aus Abschnitt 4.6 verwendet. Die Knotenzahl liegt zwischen 80 und 120.

FA: Finale Bilder analysieren

4.9. Best Solver

In diesem Experiment wird geprüft, welche Instanzen der Solver lösen kann, wenn das Zeitlimit von 5 Minuten auf 30 Minuten erhöht wird. Dafür wurden Instanzen mit zunehmender Größe generiert, bis der Solver diese nicht mehr lösen konnte. Die Knotenzahl lag am Ende zwischen 130 und 200. Es wird erneut der beste Solver aus Abschnitt 4.6 verwendet.

FA: Finale Bilder analysieren

4.10. Zielfunktion

FA: @Michael du meinstest ich soll hier die Bound mit im Diagramm anzeigen. Ich weiß grade nicht wie ich das machen soll da er auf einer Achse mit Prozent nicht sinnvoll angezeigt werden kann

FA: Wie soll ich die Diagramme aufbauen? Ich habe akutell Nur Greedy 120 Knoten instanzen und dann jetzt alle vier als eigene Diagramme.

In diesem Experiment werden die drei verschiedenen Optimierungsansätze verglichen. Erste Ansatz ist der Flips Ansatz, die anderen beiden Ansätze sind die Zielfunktionen von OrTools

Der Flips Ansatz schafft Instanzen mit Maximal 8 Knoten. Dementsprechend wird er im weiteren nicht betrachtet. Alle anderen Durchläufe in diesem Experiment wurden mit 120 Knoten durchgeführt.

Im folgenden werden Ja-Instanzen und Nein-Instanzen getrennt betrachtet. Bei Ja-Instanzen ist interessant wie schnell die Instanze gelöst werden kann und ab wann gute Ergebnisse erzielt werden. Bei den Nein-Instanzen ist hingegen interessanter wie nah der Solver an eine Lösung kommt.

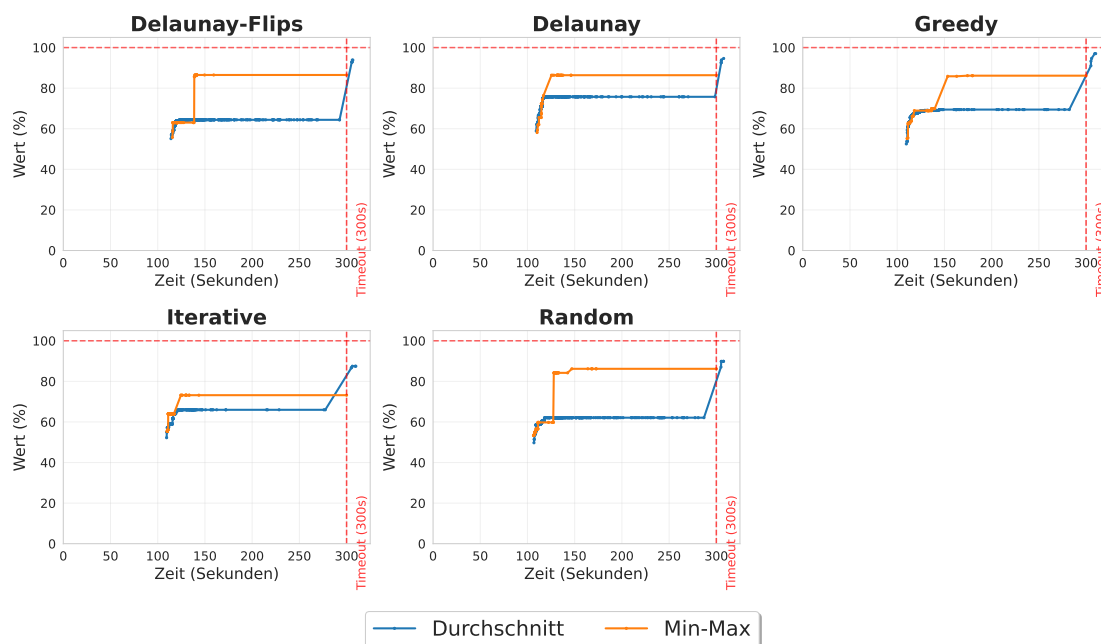


Abbildung 4.4.: eval11 move

- Min Max bei Ja besser
- Durchschnitt bei Nein besser

Abbildung D.1 ?? Abbildung 4.4 ??

4. Auswertung

4.11. Propagation CDCL

Der CDCL Solver namens *CaDiCaL* [4] von Armin Biere ermöglicht es, eine neue Entscheidungsebene mit einer eigenen Funktion zu erzeugen. Diese Funktion kann dann das nächste Literal setzen. Im Folgenden wird diese Funktion *Propagation* genannt. Die Idee bei dieser *Propagation* ist es, die Bedingungen aus Abschnitt 3.2 auf Teilinstanzen anzuwenden.

4.11.1. Visualisierung

Der erste Schritt war es, die Teilinstanzen, mit denen die *Propagation* aufgerufen wird, zu visualisieren.

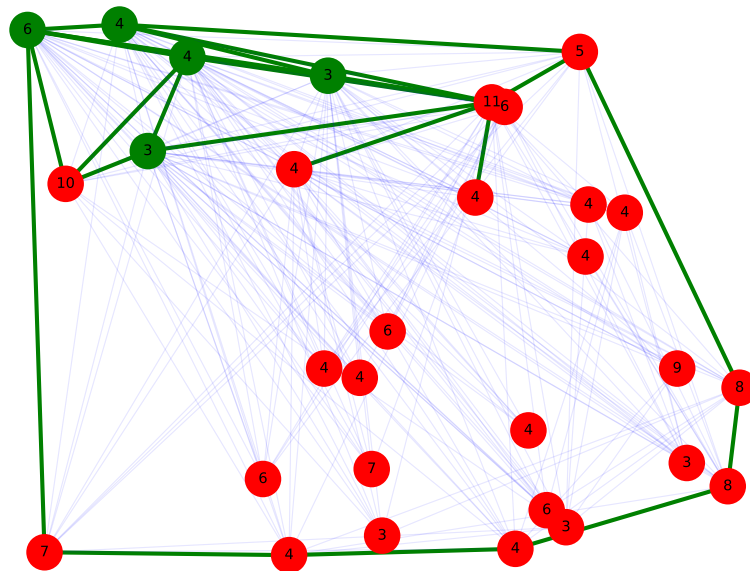


Abbildung 4.5.: Eine Teilinstanz bei einem *Propagation* Aufruf. Blau auf false gesetzt, grün auf true. Grüner Knoten Sollgrad erfüllt. Zahl in Knoten ist Sollgrad. TODO schön aufschreiben mit finalem Bild und erklären das von rand ausgehend fertig

In Abbildung 4.5 ist eine beispielhafte Teilinstanz dargestellt. Bei der Visualisierung wurde festgestellt, dass es häufig zwei Arten von Teilinstanzen gibt. Die erste Variante

bildet ein Konstrukt, das eine Verbindung zum Rand besitzt. In diesem Konstrukt ist meist eine gültige gradbeschränkte Triangulierung vorhanden. Die zweite Variante besteht aus einzelnen Kanten, die isoliert in einer Fläche liegen.

4.11.2. Implementierung

Die Idee der *Propagation* besteht darin, die in Abschnitt 3.2 beschriebenen Bedingungen zum Halbieren der Instanz anzuwenden. Die *Propagation* ist dabei insbesondere für die erste Variante der Teilinstanzen optimiert. Der Vorteil dieser Teilinstanzen ist, dass die gesamte Instanz bereits teilweise halbiert ist. Daher können gezielt Kanten getestet werden, die diese Halbierung vervollständigen.

Es wird angenommen, dass die Hülle fixiert wurde und alle Kanten, die die gesamte Instanz halbieren, bereits getestet wurden. Der Grund hierfür ist, dass die Hülle eine geeignete Fläche bietet, mit der alle Punkte betrachtet werden können.

Zur Umsetzung wird mithilfe eines Arrangements verfolgt, welche Kanten aktuell aktiv sind. Es ist darauf zu achten, dass bei jeder neuen Entscheidungsebene das Arrangement gespeichert und eindeutig einer Ebene zugeordnet wird. Tritt ein Konflikt auf, wird das Arrangement auf die entsprechende Entscheidungsebene zurückgesetzt.

Wird die *Propagation* aufgerufen, können alle Flächen des Arrangements betrachtet werden. Relevant sind dabei nur Flächen, die mehr als drei Kanten besitzen. Zusätzlich kann die unbegrenzte Fläche ausgeschlossen werden, da alle Punkte durch die Hülle eingeschlossen sind. Für diese Flächen können dann alle teilenden Kanten erzeugt werden, indem sämtliche Zweierkombinationen der Randknoten betrachtet werden. Dadurch wird die Fläche in zwei Hälften geteilt und die Kante kann getestet werden.

Vor dem Testen können alle Kanten übersprungen werden, die bereits festgelegt wurden oder keine gültigen Kanten sind. Für die Punktzuordnung kann zu Beginn jeder *Propagation* jeder Punkt einer Fläche zugeordnet werden. Sollte dann eine Kante getestet werden, können alle Punkte der zugehörigen Fläche betrachtet und ähnlich wie in Abschnitt 3.2.1 mithilfe von *Rechtsknicks* und *Linksknicks* einer Teilhälfte zugeordnet werden.

Führt das Testen einer Kante zu einem Konflikt, kann das zugehörige Literal l auf *falsch* gesetzt werden. Als Begründung kann folgende Klausel verwendet werden

$$(\bar{l} \vee \bigvee_{e \in E_A} \bar{x}_e)$$

wobei E_A die Menge der aktiven Kanten bezeichnet.

4.11.3. Experiment

In diesem Experiment wird betrachtet wie viele Erfolgreiche *Propagation* also jene, welche eine Kante ausschließen können, in einer Instanz möglich sind. Dafür werden alle Erfolgreichen Aufrufe gezählt und diese Zahl durch die Anzahl der *Propagation* Aufrufe geteilt. Dabei wird in diesem Experiment nicht die Laufzeit gemessen da durch die vielen Kanten, die getestet werden, die Laufzeit stark beeinflusst wird.

4. Auswertung

Eine *Propagation* soll im normalfall sehr schnell entscheiden welche Kante als nächstes betrachtet wird. In diesem Experiment geht es nur darum zu bestimmen ob es sinnvoll ist eine *Propagation* mit diesem Ansatz zu nutzen. Bei diesem Experiment wurden Instanzen der gröÙe 30 bis 60 Knoten betrachtet. Das Experiment wurde auf dem Ryzen 9 durchgeführt.

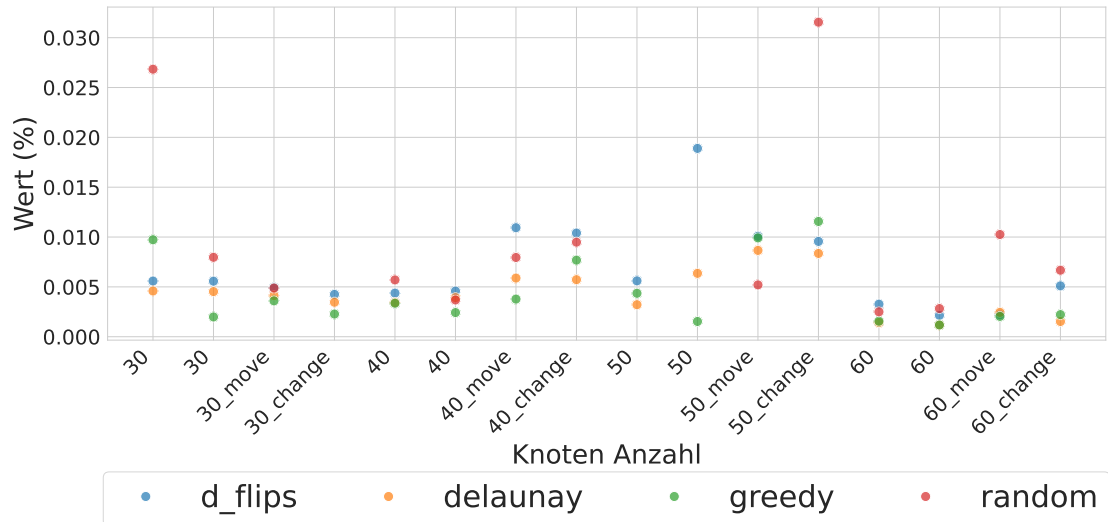


Abbildung 4.6.: Anzahl propagation cdcl

FA: Größere Instanzen laufen lassen, wahrscheinlich nicht viel größere da es keinen Timeout gibt und das das Testen schwierig macht

In Abbildung 4.6 sind diese Relativen Anzahlen pro Instanz zu sehen. Dabei fällt auf das vorallem Random Instanzen sehr viele Erfolgreiche *Propagation* haben. Greedy und Delaunay hingegen haben nur wenige Erfolgreiche *Propagation*. Nicht ganz deutlich aber zu erkennen ist das Nein-Instanzen mehr Erfolgreiche *Propagation* haben als Ja-Instanzen. Bei diesem Muster kann es sich aber auch um einen Zufall handeln da nur wenige verschiedene Instanzen betrachtet wurden.

4.12. Instanzen Schwierigkeit

In diesem Abschnitt wird Probiert die Schwierigkeit der Instanzen zu bestimmen. Aus den vorigen Experimenten ist bekannt das *Delaunay* und *Greedy* die schwereren Instanzen sind, *Delaunay Flips* vor *Random* im Mittelfeld und *Iterativ* die einfachste Instanz ist. Im ersten Teil wird geschaut ob die Anzahl der Lösungen pro Instanz einen Einfluss auf die Schwierigkeit hat. Danach im zweiten Teil wird geschaut wie der Grad verteilt ist. Also wie oft jeder Grad vertreten wird. Im Letzen Teil wird geschaut ob ein Zusammenhang zu den Kantenlängen besteht. Für alle drei Teile werden Instanzen der gröÙe 30 bis 120 Knoten betrachtet.

4.12.1. Anzahl Lösungen

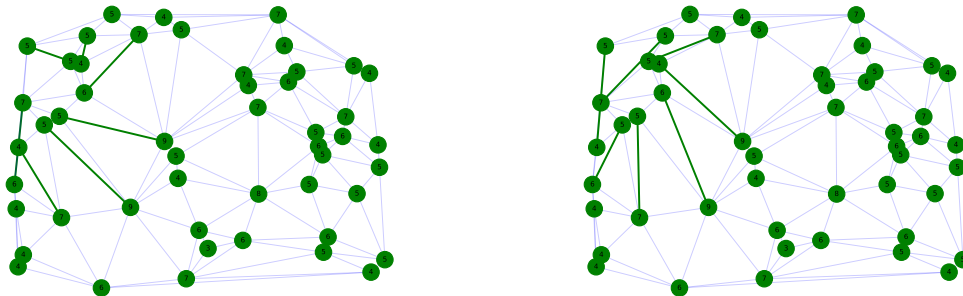


Abbildung 4.7.: In dem Bild sind zwei verschiedene Lösungen für eine Instanz zu sehen. In Grün sind die Kanten zu sehen welche in der Instanz unterschiedlich sind. In Blau die gleichen Kanten. Die Zahlen in den Knoten geben den Sollgrad an.

In table C.1 ist zu sehen das fast alle Instanzen nur eine Lösungen haben. Vorallem ist aber zu sehen das es keinen Zusammenhang zwischen der Anzahl der Lösungen und der Instanze Art gibt. Es kann also davon ausgegangen werden das die Anzahl der Lösungen keinen Einfluss auf die Schwierigkeit der Instanzen hat. Interessanter weißte ist zu sehen das die Kanten welche in einer Instanze unterschiedliche sind klar einem Bereich zugeordnet werden könne. Wie man in ?? sieht, sind die Kanten in der linken Hälfte der Instanz unterschiedlich, während die rechte Hälfte keine Unterschiede aufweist. Man könnte also die Grade welche mehrere Lösungen zulassen einem Lokalen Bereich zuordnen.

4.12.2. Grad Verteilung

In diesem Experiment wird Probiert anhand der Gradverteilung der Knoten zu erkennen ob es einen Zusammenhang zwischen der Schwierigkeit und dem Grad gibt. Dafür werden die Ja-Instanzen der Größe 30 bis 120 Knoten betrachtet. In Abbildung 4.8 ist die Gradverteilung der Knoten zu sehen. Auf den ersten Blick lässt sich vermuten das es einen Zusammenhang zwischen der Schwierigkeit und dem Grad gibt. Es ist zu erkennen das Vorallem Random und Iterativ eine Rechtsschiefe Verteilung haben. Dazu passend haben Delaunay und Greedy eher eine zentrierte Verteilung. Auch das Delaunay-Flips Rechtsschiefer als Delaunay und Greedy ist würde passen, da Delaunay-Flips einfacher als Delaunay sind. Es lässt sich also Anhand dieser Beispiel vermuten das einfachere Instanzen eher kleinere Grade haben und schwierigere Instanzen mehr Verteile mittlere Grade.

4. Auswertung

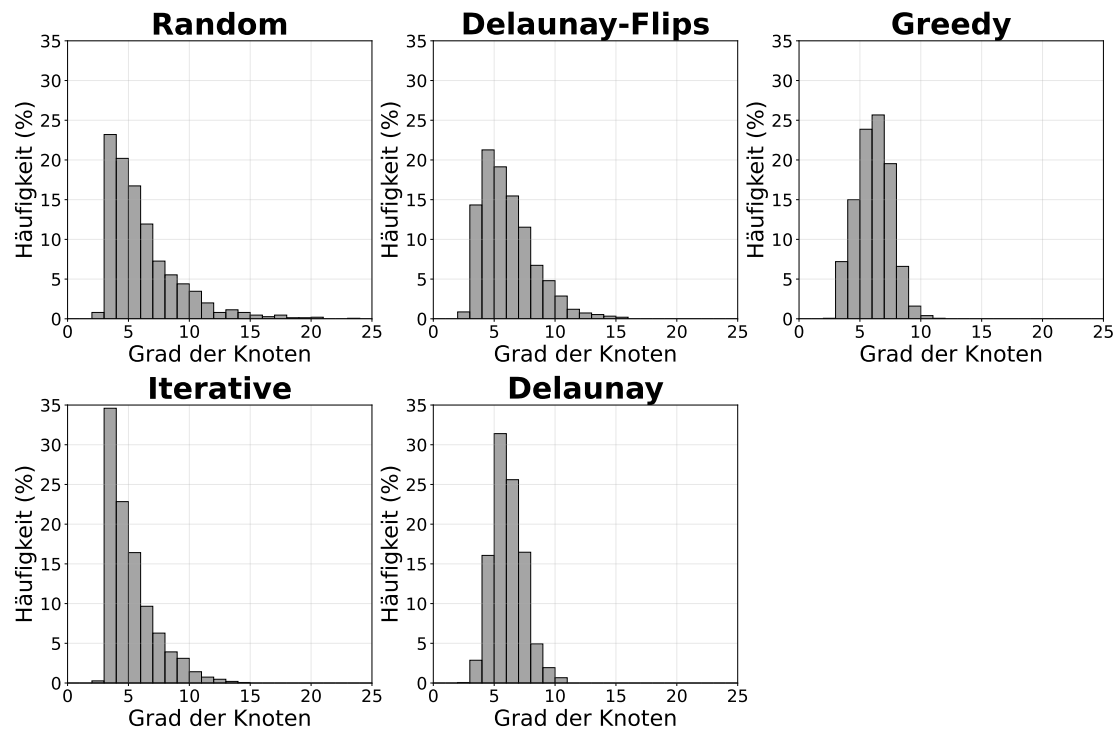


Abbildung 4.8.: Eval8 Anzahl Lösungen Greedy

4.12.3. Kantenlängen Verteilung

In diesem Experiment wird ebenfalls probiert einen Zusammenhang zwischen der Schwierigkeit und einer Eigenschaft zu finden. In diesem Fall werden die Kantenlängen betrachtet. Allerdings müssen diese Normiert werden da sonst durch weit auseinander liegende Knoten die Verteilung verzerren würden. Die Normierung erfolgt durch das Teilen durch die maximale Kantenlänge. Das heißt für jede Instanz wird die maximale Kantenlänge bestimmt und alle Kantenlängen durch diese geteilt. Für dieses Experiment wurden nur Ja-Instanzen der Größe 30 bis 80 Knoten betrachtet.

Auch hier lässt sich eine Verbindung zwischen der Schwierigkeit und der Verteilung erkennen. Leichtere Instanzen haben eher lange Kanten als schwere Instanzen. So sind bei schweren Instanzen wie Greedy und Delaunay die Kantenlängen eher links also bei den kurzen Kanten angesiedelt. Leichte Instanzen wie Iterativ hingegen haben deutlich weniger kurze Kante und dafür mehr längere Kanten.

4.12.4. Schwierigkeit Zusammenfassung

In diesem Abschnitt wurde festgestellt dass die Anzahl der Lösungen keinen Einfluss auf die Schwierigkeit hat. Die Gradverteilung und die Kantenlängenverteilung hingegen scheinen einen Einfluss auf die Schwierigkeit zu haben. Hierbei deuten viele kurze Kanten und verteilte Grade auf eine schwere Instanz hin. Hingegen verteilte längere Kanten und wenige

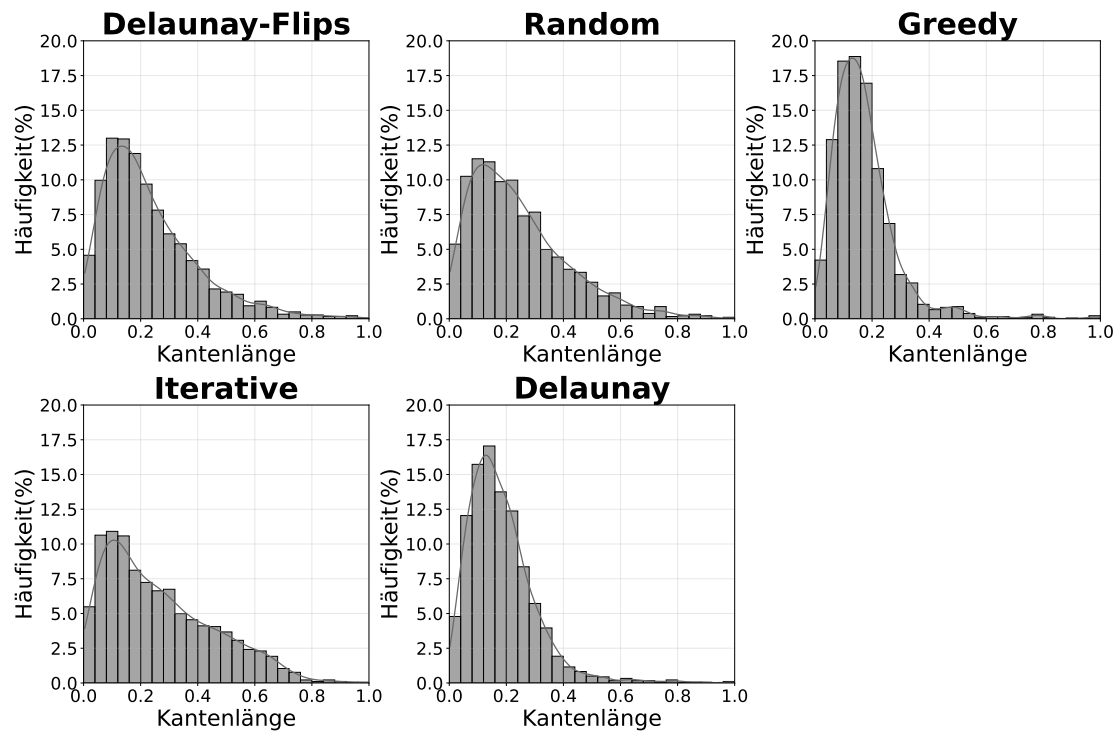


Abbildung 4.9.: Eval8 Kanten verteilung

hohe Grade auf eine leichte Instanz.

5. Conclusion (and Future Work)

This is the end of your thesis! Give a summary of what you did.

You can also write down possible future work you or other people could look at.

- meine Random instanzen waren nicht normalverteilt
- Könnte man in zukunft anschauen
- gibt Paper mit einfachen Polygone mit normalverteilter Triangulierung

Literaturverzeichnis

- [1] Roberto Asín, Robert Nieuwenhuis, Albert Oliveras, and Enric Rodríguez-Carbonell. Cardinality networks and their applications. In *International Conference on Theory and Applications of Satisfiability Testing*, pages 167–180. Springer, 2009.
- [2] Olivier Bailleux and Yacine Boufkhad. Efficient cnf encoding of boolean cardinality constraints. In *International conference on principles and practice of constraint programming*, pages 108–122. Springer, 2003.
- [3] Kenneth E Batchner. Sorting networks and their applications. In *Proceedings of the April 30–May 2, 1968, spring joint computer conference*, pages 307–314, 1968.
- [4] Armin Biere. Arminbiere/cadical: Cadical sat solver, 2019. URL: <https://github.com/arminbiere/cadical>.
- [5] Prosenjit Bose and Ferran Hurtado. Flips in planar graphs. *Computational Geometry*, 42(1):60–80, 2009. URL: <https://www.sciencedirect.com/science/article/pii/S0925772108000370>, doi:10.1016/j.comgeo.2008.04.001.
- [6] Basudeb Datta and Subhojoy Gupta. Degree-regular triangulations of surfaces. *arXiv preprint arXiv:1711.01247*, 2017.
- [7] Ana Maria de Almeida, Pedro Martins, and Maurício C de Souza. Min-degree constrained minimum spanning tree problem: complexity, properties, and formulations. *International Transactions in Operational Research*, 19(3):323–352, 2012.
- [8] Mark de Berg, Marc van Kreveld, Mark Overmars, and Otfried Cheong Schwarzkopf. *Delaunay Triangulations*, pages 183–210. Springer Berlin Heidelberg, Berlin, Heidelberg, 2000. doi:10.1007/978-3-662-04245-8_9.
- [9] Sándor P Fekete, Phillip Keldenich, and Michael Perk. Exact algorithms for minimum dilation triangulation. *arXiv preprint arXiv:2502.18189*, 2025.
- [10] Sándor P Fekete, Samir Khuller, Monika Klemmstein, Balaji Raghavachari, and Neal Young. A network-flow technique for finding low-weight bounded-degree spanning trees. *Journal of Algorithms*, 24(2):310–324, 1997.
- [11] Laxmi P Gewali and Bhaikaji Gurung. Degree aware triangulation of annular regions. In *Information Technology-New Generations: 15th International Conference on Information Technology*, pages 683–690. Springer, 2018.

- [12] F. Hurtado, M. Noy, and J. Urrutia. Flipping edges in triangulations. In *Proceedings of the Twelfth Annual Symposium on Computational Geometry*, SCG '96, page 214–223, New York, NY, USA, 1996. Association for Computing Machinery. doi:10.1145/237218.237367.
- [13] J. Knowles and D. Corne. A new evolutionary approach to the degree-constrained minimum spanning tree problem. *IEEE Transactions on Evolutionary Computation*, 4(2):125–134, 2000. doi:10.1109/4235.850653.
- [14] Donald E. Knuth. *The Art of Computer Programming, Volume 4, Fascicle 6: Satisfiability*. Addison-Wesley Professional, 1st edition, 2015.
- [15] Mohan Krishnamoorthy, Andreas T. Ernst, and Yazid M. Sharaiha. Comparison of algorithms for the degree constrained minimum spanning tree. *Journal of Heuristics*, 7(6):587–611, 2001. doi:10.1023/A:1011977126230.
- [16] Antonio Morgado, Alexey Ignatiev, and Joao Marques-Silva. Mscg: Robust core-guided maxsat solving: System description. *Journal on Satisfiability, Boolean Modelling and Computation*, 9(1):129–134, 2014.
- [17] Oleg R Musin. Properties of the delaunay triangulation. In *Proceedings of the thirteenth annual symposium on Computational geometry*, pages 424–426, 1997.
- [18] Toru Ogawa, Yangyang Liu, Ryuzo Hasegawa, Miyuki Koshimura, and Hiroshi Fujita. Modulo based cnf encoding of cardinality constraints and its application to maxsat solvers. In *2013 IEEE 25th International Conference on Tools with Artificial Intelligence*, pages 9–17. IEEE, 2013.
- [19] Micha Sharir, Adam Sheffer, and Emo Welzl. On degrees in random triangulations of point sets. In *Proceedings of the twenty-sixth annual symposium on Computational geometry*, pages 297–306, 2010.
- [20] Carsten Sinz. Towards an optimal cnf encoding of boolean cardinality constraints. In *International conference on principles and practice of constraint programming*, pages 827–831. Springer, 2005.

A. SAT Degree Auswertung

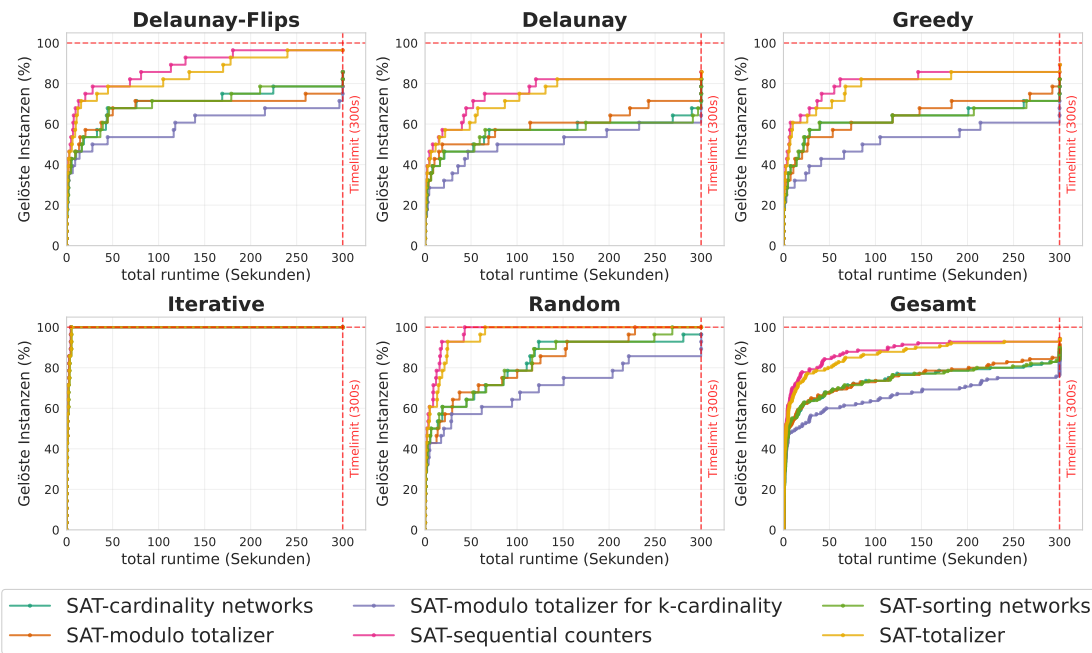


Abbildung A.1.: Die *exact* Methode für die Gradbeschränkung in einem SAT Solver, wurde mit verschiedenen Kardinalitätsbedingung getestet.

A. SAT Degree Auswertung

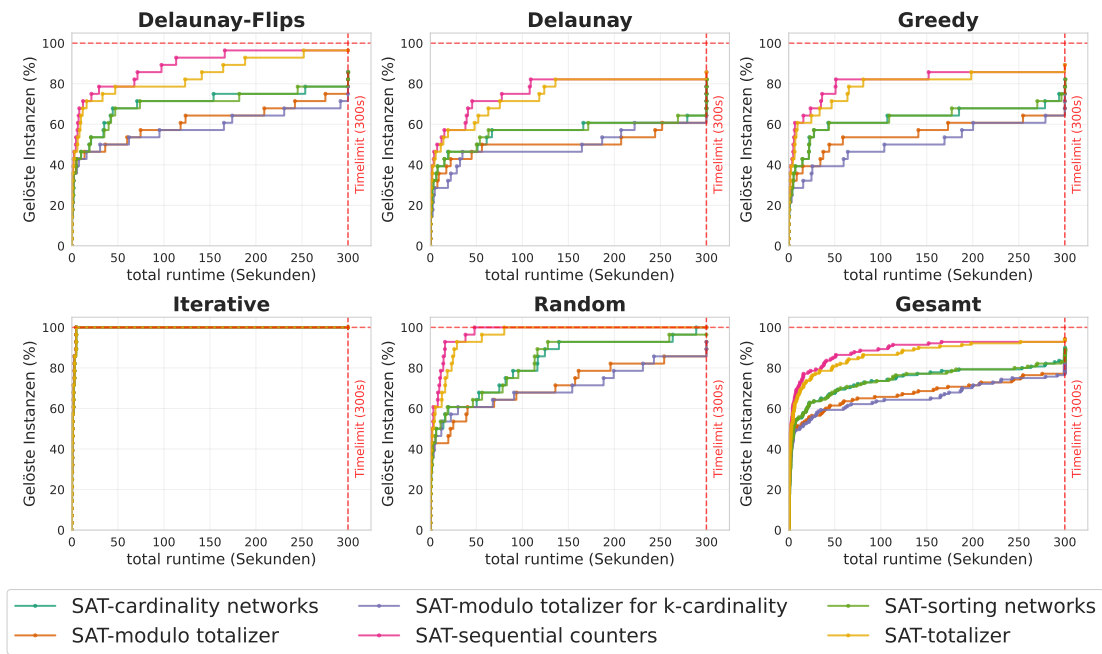


Abbildung A.2.: Die *atleast* Methode für die Gradbeschränkung in einem SAT Solver, wurde mit verschiedenen Kardinalitätsbedingung getestet.

B. Alle Parameter

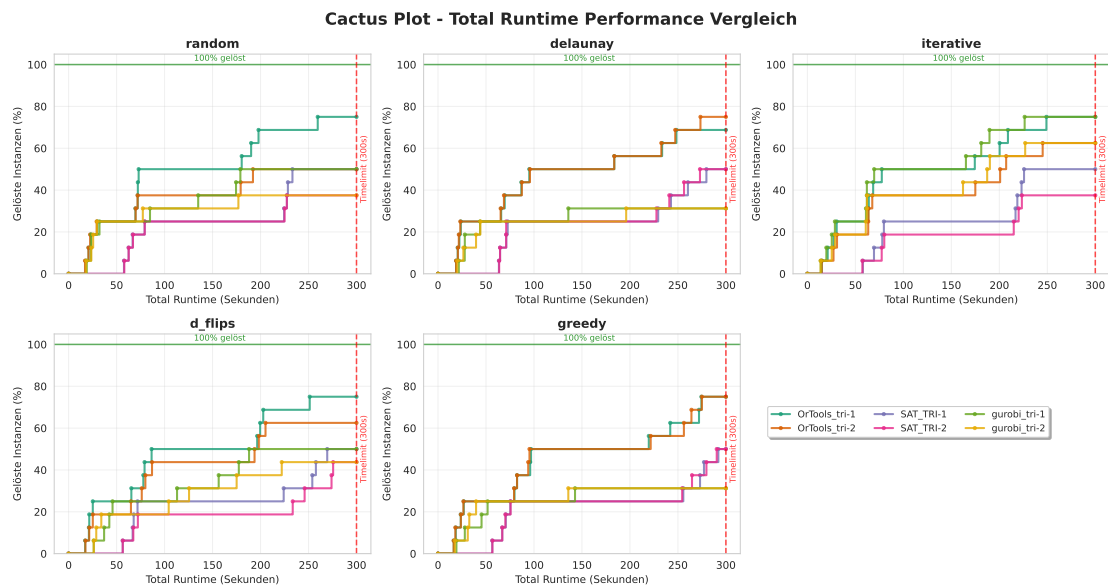


Abbildung B.1.: Eval1 Dreiecke

B. Alle Parameter

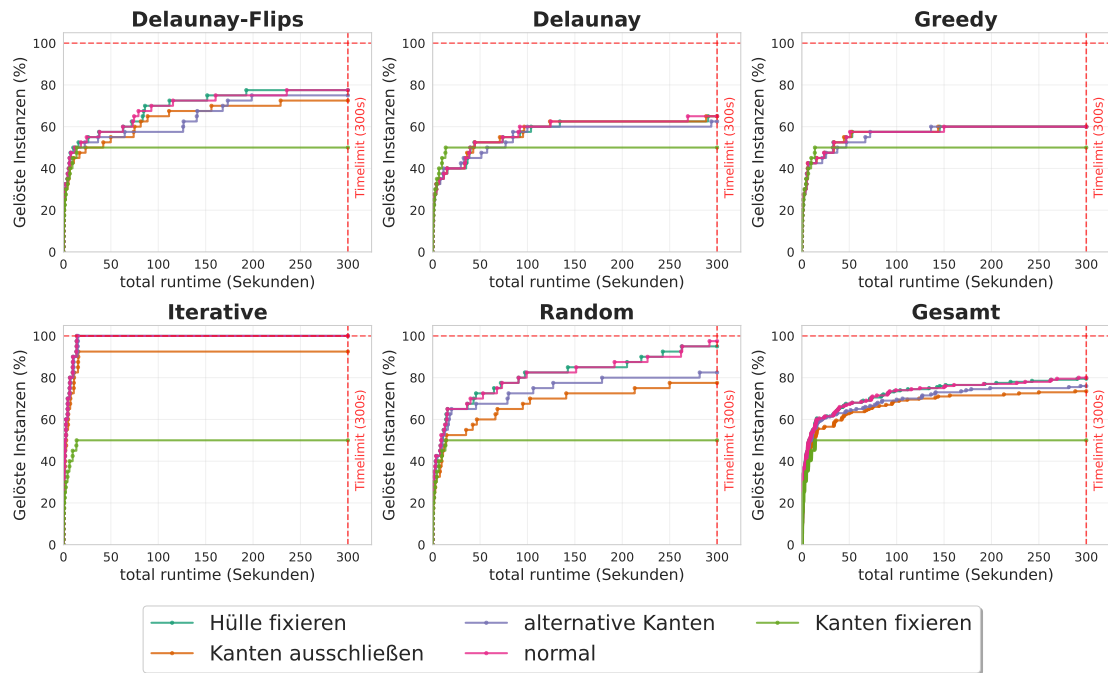


Abbildung B.2.: Eval1 Sat

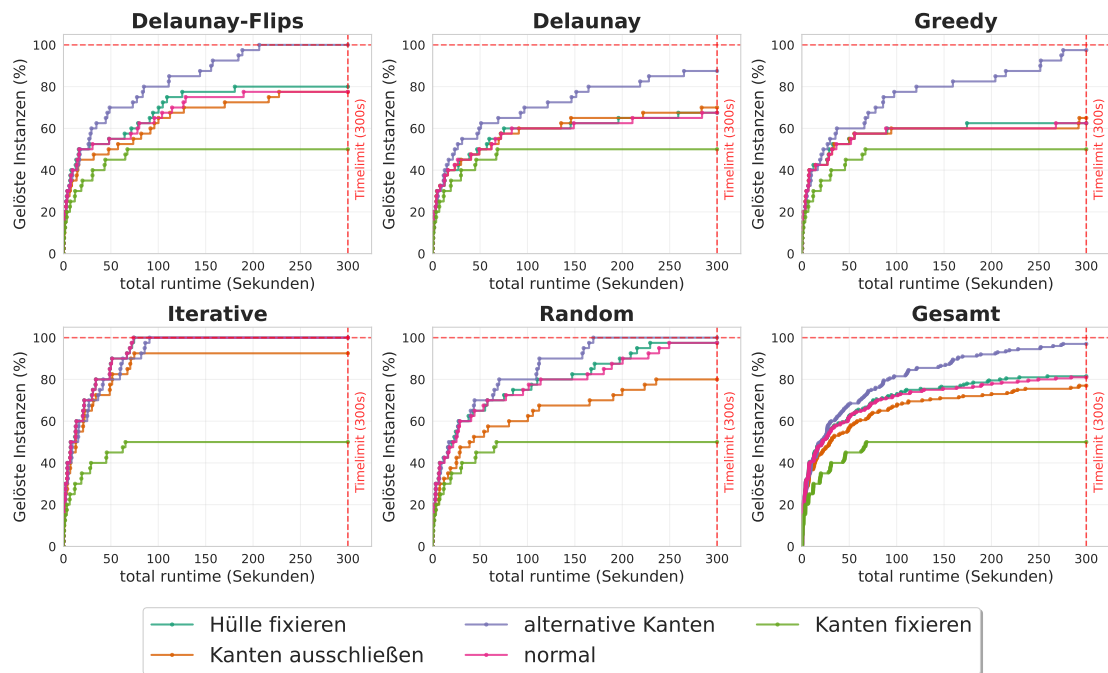


Abbildung B.3.: Eval1 OrTools

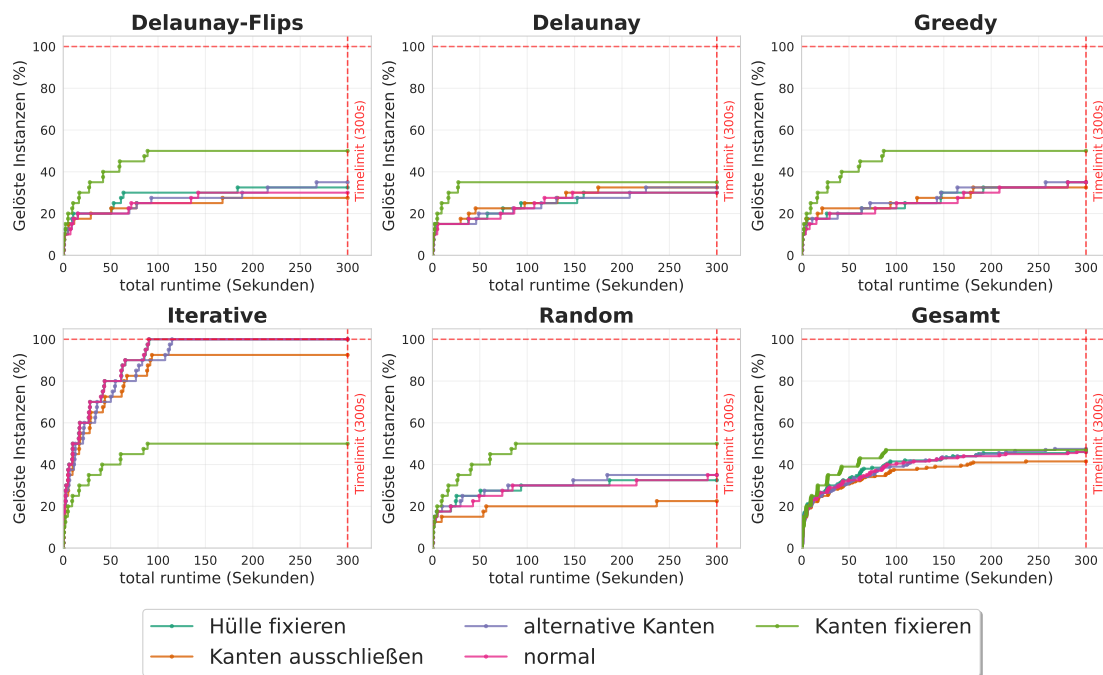


Abbildung B.4.: Eval1 Gurobi

C. Anzahl Lösungen

Knoten Anzahl	Delaunay Flips	Delaunay	Greedy	Iterative	Random
30_1	1	1	1	1	1
30_2	1	1	1	1	1
40_1	1	1	1	1	1
40_2	1	1	1	1	1
50_1	1	2	1	1	1
50_2	1	1	1	1	1
60_1	1	1	1	1	1
60_2	1	1	1	1	1
70_1	1	1	1	1	1
70_2	1	1	1	1	1
80_1	1	1	1	1	1
80_2	1	1	1	1	1
90_1	1	-1	1	1	1
90_2	1	-1	1	1	1
100_1	1	-1	-1	1	1
100_2	1	-1	-1	1	1
110_1	-1	-1	-1	1	1
110_2	-1	-1	-1	1	1
120_1	-1	-1	-1	1	1
120_2	-1	-1	-1	1	1

Tabelle C.1.: In dieser Tabelle werden die Anzahl der Lösungen für die verschiedenen Ansätze dargestellt. In der ersten Spalte werden die Anzahl der Knoten der Instanz angegeben. Hierbei wurde jede Knotengröße zwei mal getestet. In den anderen Spalten sind jeweils die Werte für eine Instanz gegeben. Die Werte -1 bedeuten, dass nach einer Stunde ein Timeout erreicht wurde und somit keine Lösung gefunden wurde konnte. Auffällig ist, dass es nur eine einzige Instanz gibt, welche zwei Lösungen besitzt.

D. Zielfunktion

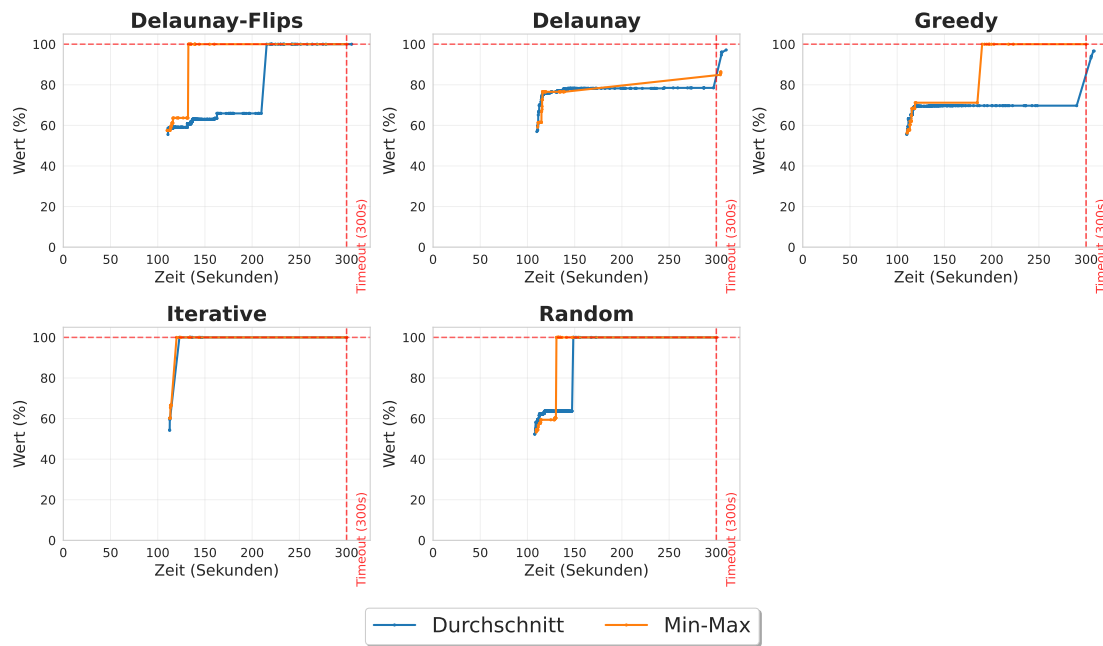


Abbildung D.1.: eval11 36